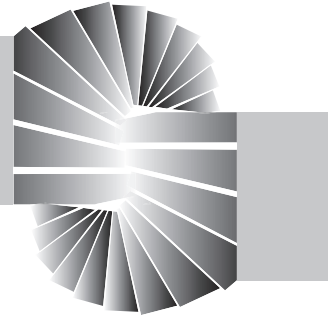

PARTE

1 2 3 4



O sistema de computação

ASSUNTOS DA PARTE 2

Um sistema de computação consiste em processador, memória, dispositivos de E/S e as interconexões entre esses componentes principais. Com a exceção do processador, que é suficientemente complexo para dedicarmos a Parte 3 ao seu estudo, a Parte 2 examina cada um desses componentes com detalhes.

MAPA DA PARTE 2

Capítulo 3 Visão de alto nível da função e interconexão do computador

No nível superior, um computador consiste em um processador, memória e componentes de E/S. O comportamento funcional do sistema consiste na troca de dados e sinais de controle entre esses componentes. Para dar suporte a essa troca, os componentes precisam ser interconectados. O Capítulo 3 começa com um breve exame dos componentes do computador e seus requisitos de entrada e saída. O capítulo então examina os principais aspectos que afetam o projeto de interconexão, especialmente a necessidade de oferecer suporte a interrupções. A maior parte do capítulo é dedicada a um estudo da abordagem mais comum da interconexão: o uso de uma estrutura de barramentos.

Capítulo 4 Memória cache

A memória do computador apresenta uma grande variedade de tipo, tecnologia, organização, desempenho e custo. O sistema de computação típico é equipado com uma hierarquia de subsistemas de memória, alguns internos (acessíveis diretamente pelo processador) e alguns externos (acessíveis pelo processador por meio de um módulo de E/S). O Capítulo 4 começa com uma visão geral dessa hierarquia. Em seguida, o capítulo trata dos detalhes do projeto da memória cache, incluindo caches de código e dados separados e caches de dois níveis.

Capítulo 5 Memória interna

O projeto de um sistema de memória principal é uma batalha sem fim entre três requisitos de projeto concorrentes: grande capacidade de armazenamento, acesso rápido e baixo custo. Com a evolução da tecnologia de memória, cada uma dessas três características está mudando, de modo que as decisões de projeto na organização da memória principal devem ser revisadas novamente a cada nova implementação. O Capítulo 5 focaliza aspectos do projeto relacionados à memória interna. Primeiro, a natureza e a organização da memória semicondutora principal é examinada. Depois, exploramos as organizações recentes da memória DRAM avançada.

Capítulo 6 Memória externa

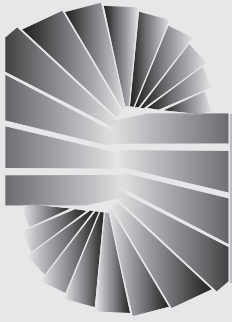
Para uma capacidade de armazenamento realmente grande e para um armazenamento mais permanente do que existe para a memória principal, uma organização de memória externa é necessária. O tipo mais usado de memória externa é o disco magnético, e grande parte do Capítulo 6 se concentra nesse assunto. Primeiro, examinamos a tecnologia e as considerações de projeto do disco magnético. Depois, explicamos o uso da organização RAID (do inglês *redundant array of independent disks*) para melhorar o desempenho da memória em disco. O Capítulo 6 também examina o armazenamento óptico e em fita.

Capítulo 7 Entrada/Saída

Módulos de E/S são interconectados com o processador e a memória principal, e cada um controla um ou mais dispositivos externos. O Capítulo 7 é dedicado a diversos aspectos da organização de E/S. Essa é uma área complexa e menos compreendida do que outras áreas do projeto de sistema de computação em termos de atender as demandas de desempenho. O Capítulo 7 examina os mecanismos pelos quais um módulo de E/S interage com o restante do sistema de computação, usando as técnicas da E/S programada, E/S de interrupção e acesso direto à memória (DMA, do inglês *direct memory access*). A interface entre um módulo de E/S e os dispositivos externos também é descrita.

Capítulo 8 Suporte ao sistema operacional

Um exame detalhado dos sistemas operacionais (OS, do inglês *operation systems*) está fora do escopo deste livro. Porém, é importante entender as funções básicas de um sistema operacional e como ele explora o hardware para oferecer o desempenho desejado. O Capítulo 8 descreve os princípios básicos dos sistemas operacionais e discute os recursos de projeto específicos no hardware de computador voltados para oferecer suporte ao sistema operacional. O capítulo começa com uma breve história, que serve para identificar os principais tipos de sistemas operacionais e para motivar seu uso. Em seguida, a multiprogramação é explicada examinando as funções de escalonamento a longo e curto prazos. Finalmente, um estudo do gerenciamento de memória inclui uma discussão sobre segmentação, paginação e memória virtual.



Visão de alto nível da função e interconexão do computador

3.1 Componentes do computador

3.2 Função do computador

- Busca e execução de instruções
- Interrupções
- Função de E/S

3.3 Estrutura de interconexão

3.4 Interconexão de barramento

- Estrutura de barramento
- Hierarquia de barramento múltiplo
- Elementos do projeto de barramento

3.5 PCI

- Estrutura de barramento
- Comandos PCI
- Transferências de dados
- Arbitração

3.6 Leitura recomendada e sites Web

- Sites web recomendados

Apêndice 3A Diagramas de sincronização

PRINCIPAIS PONTOS

- Um ciclo de instrução consiste em uma busca de instrução, seguida por zero ou mais buscas de operandos, seguidas por zero ou mais armazenamentos de operandos, seguidos por uma verificação de interrupção (se as interrupções estiverem habilitadas).
- Os principais componentes do sistema de computação (processador, memória principal, módulos de E/S) precisam ser interconectados a fim de trocar dados e sinais de controle. O meio de interconexão mais popular é o uso de um barramento do sistema compartilhado, consistindo em múltiplas linhas. Nos sistemas contemporâneos, normalmente existe uma hierarquia de barramentos para melhorar o desempenho.
- Os principais elementos de projeto para os barramentos incluem arbitração (a permissão para enviar sinais nas linhas do barramento pode ser controlada de forma central ou distribuída); temporização (os sinais no barramento podem ser sincronizados com um clock central ou enviados de forma assíncrona com base na transmissão mais recente); e largura (número de linhas de endereço e número de linhas de dados).

No nível superior, um computador consiste em CPU, memória e componentes de E/S, com um ou mais módulos de cada tipo. Esses componentes são interconectados de alguma forma para realizar a função básica do computador, que é executar programas. Assim, em um nível mais alto, podemos descrever um sistema de computação (1) descrevendo o comportamento externo de cada componente — ou seja, os dados e sinais de controle que ele troca com outros componentes; e (2) descrevendo a estrutura de interconexão e os controles exigidos para gerenciar o uso da estrutura de interconexão.

Essa visão de alto nível da estrutura e da função é importante devido ao seu poder explicativo na compreensão da natureza de um computador. Igualmente importante é o seu uso para entender as questões cada vez mais complexas da avaliação de desempenho. Um conhecimento da estrutura e função de alto nível gera compreensão dos gargalos do sistema, caminhos alternativos, a magnitude de falhas do sistema caso um componente falhe e a facilidade de acrescentar melhorias de desempenho. Em muitos casos, os requisitos para maior poder do sistema e capacidades à prova de falhas estão sendo atendidos pela mudança do projeto, em vez de simplesmente aumentar a velocidade e a confiabilidade dos componentes individuais.

Este capítulo enfoca as estruturas básicas utilizadas para a interconexão dos componentes do computador. Ele começa com uma rápida explicação dos componentes básicos e seus requisitos de interface. Depois, vemos um panorama funcional e, em seguida, somos preparados para examinar o uso de barramentos para interconectar os componentes do sistema.



3.1 Componentes do computador

Conforme discutimos no Capítulo 2, praticamente todos os projetos de computadores modernos são baseados em conceitos desenvolvidos por John Von Neumann no *Institute for Advanced Studies*, em Princeton. Esse projeto é conhecido como *arquitetura de Von Neumann* e é baseado em três conceitos principais:

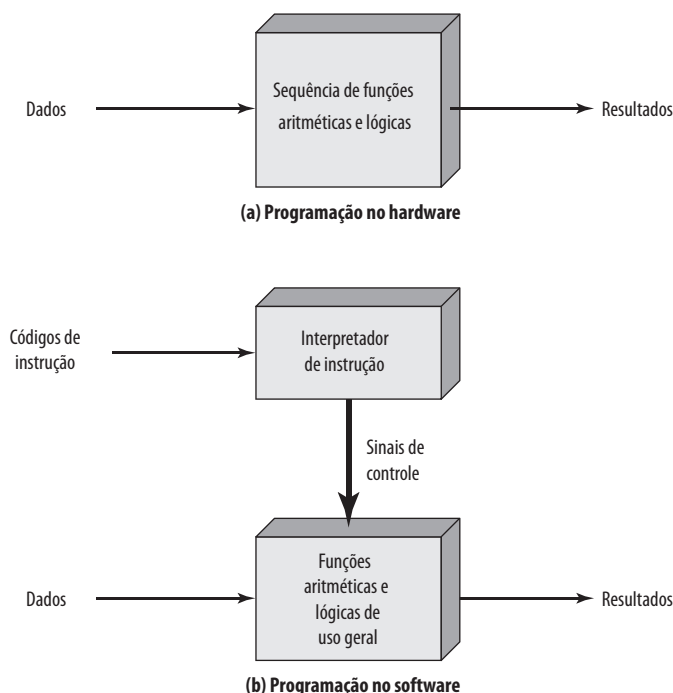
- Dados e instruções são armazenados em uma única memória de leitura e escrita.
- O conteúdo dessa memória é endereçável por local, sem considerar o tipo de dados neles contido.
- A execução ocorre em um padrão sequencial (a menos que modificado explicitamente) de uma instrução para a seguinte.

O raciocínio por trás desses conceitos foi discutido no Capítulo 2, mas merece ser resumido aqui. Existe um pequeno conjunto de componentes lógicos básicos que podem ser combinados de diversas maneiras para armazenar dados binários e realizar operações aritméticas e lógicas sobre esses dados. Se houver um cálculo em particular a ser realizado, uma configuração de componentes lógicos projetados especificamente para esse cálculo poderia ser construída. Podemos pensar no processo de conexão dos vários componentes na configuração desejada como uma forma de programação. O “programa” resultante está na forma de hardware e é chamado de *programa hardwired*.

Agora, considere esta alternativa: suponha que queremos construir uma configuração de uso geral das funções aritméticas e lógicas. Esse conjunto de hardware realizará diversas funções sobre os dados, dependendo dos sinais de controle aplicados ao hardware. No caso original do hardware customizado, o sistema aceita dados e produz resultados (Figura 3.1a). Com o hardware de uso geral, o sistema aceita dados e sinais de controle e produz resultados. Assim, em vez de religar o hardware para cada novo programa, o programador simplesmente precisa fornecer um novo conjunto de sinais de controle.

Como os sinais de controle devem ser fornecidos? A resposta é simples, porém sutil. O programa inteiro, na realidade, é uma sequência de etapas. Em cada etapa, alguma operação aritmética ou lógica é realizada sobre alguns dados. Para cada etapa, um novo conjunto de sinais de controle é necessário. Vamos oferecer um código exclusivo para cada conjunto possível de sinais de controle, e vamos acrescentar ao hardware de uso geral um segmento que pode aceitar um código e gerar sinais de controle (Figura 3.1b).

Figura 3.1 Abordagens de hardware e software



A programação agora é muito mais fácil. Em vez de religar o hardware para cada novo programa, tudo o que precisamos fazer é oferecer uma nova sequência de códigos. Cada código, com efeito, é uma instrução, e parte do hardware interpreta cada instrução e gera sinais de controle. Para distinguir esse novo método de programação, uma sequência de códigos ou instruções é chamada de *software*.

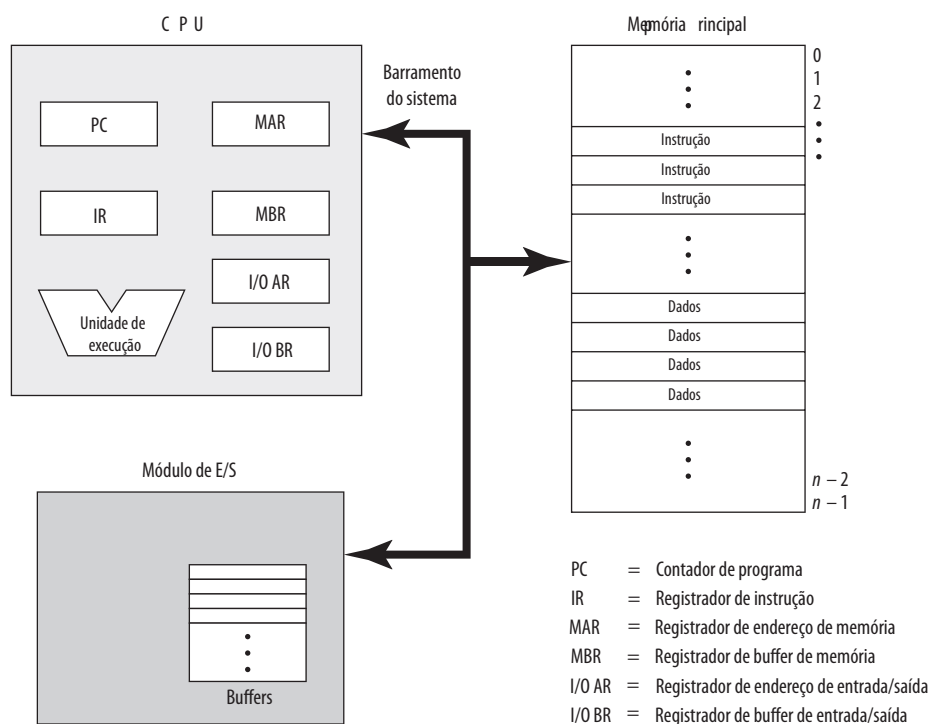
A Figura 3.1b indica dois componentes importantes do sistema: um interpretador de instrução e um módulo para funções aritméticas e lógicas de uso geral. Esses dois constituem a CPU. Vários outros componentes são necessários para resultar em um computador funcionando. Dados e instruções precisam ser colocados no sistema e para isso, precisamos de algum tipo de módulo de entrada. Esse módulo contém componentes básicos para aceitar dados e instruções em alguma forma e convertê-los para uma forma interna de sinais que possam ser usados pelo sistema. Também é necessário um meio de informar resultados, e este tem a forma de um módulo de saída. Juntos, estes são chamados de *componentes de E/S*.

Mais um componente é necessário: um dispositivo de entrada que trará dados e instruções sequencialmente. Mas um programa não é invariavelmente executado de forma sequencial; ele pode saltar (por exemplo, a instrução jump do IAS). De modo semelhante, as operações sobre dados podem exigir acesso a mais do que apenas um elemento de cada vez em uma sequência predeterminada. Assim, deverá haver um lugar para armazenar instruções e dados temporariamente. Esse módulo é chamado de *memória*, ou *memória principal*, para distingui-la do armazenamento externo, ou dispositivos periféricos. Von Neumann indicou que a mesma memória poderia ser usada para armazenar tanto instruções quanto dados.

A Figura 3.2 ilustra esses componentes de alto nível e sugere as interações entre eles. A CPU troca dados com a memória. Para essa finalidade, ela normalmente utiliza dois registradores internos (à CPU): um registrador de endereço de memória (MAR), que especifica o endereço na memória para a próxima leitura ou escrita, e um registrador de buffer de memória (MBR), que contém os dados a serem escritos na memória ou recebe os dados lidos da memória. De modo semelhante, um registrador de endereço de E/S (I/O AR) especifica um dispositivo de E/S em particular. Um registrador de buffer de E/S (I/O BR) é usado para a troca de dados entre um módulo de E/S e a CPU.

Um módulo de memória consiste em um conjunto de locais, definidos por endereços numerados sequencialmente. Cada local contém um número binário que pode ser interpretado como uma instrução ou um dado. Um

Figura 3.2 Componentes do computador: visão de alto nível



módulo de E/S transfere dados dos dispositivos externos para a CPU e a memória, e vice-versa. Ele contém buffers internos para manter esses dados temporariamente, até que possam ser enviados.

Tendo examinado rapidamente esses principais componentes, agora, vamos passar a uma visão geral de como esses componentes funcionam juntos para executar programas.



3.2 Função do computador

A função básica realizada por um computador é a execução de um programa, que consiste em um conjunto de instruções armazenadas na memória. O processador faz o trabalho real executando instruções especificadas no programa. Esta seção oferece uma visão geral dos principais elementos da execução do programa. Em sua forma mais simples, o processamento de instrução consiste em duas etapas: o processador lê (*busca*) instruções da memória, uma de cada vez, e executa cada instrução. A execução do programa consiste em repetir o processo de busca e execução de instrução. A execução da instrução pode envolver diversas operações e depende da natureza da instrução (ver, por exemplo, a parte inferior da Figura 2.4).

O processamento exigido para uma única instrução é chamado de *ciclo de instrução*. Usando a descrição simplificada em duas etapas dadas anteriormente, o ciclo de instrução é representado na Figura 3.3. As duas etapas são conhecidas como *ciclo de busca* e *ciclo de execução*. A execução do programa só termina se a máquina for desligada, se houver algum tipo de erro irreversível ou se for encontrada uma instrução do programa que interrompa o computador.



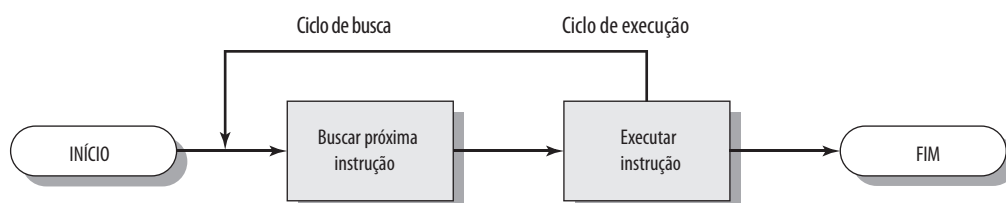
Busca e execução de instruções

No início de cada ciclo de instrução, o processador busca uma instrução da memória. Em um processador típico, um registrador chamado contador de programa (PC) mantém o endereço da instrução a ser buscada em seguida. A menos que seja solicitado de outra maneira, o processador sempre incrementa o PC após cada busca de instrução, de modo que buscará a próxima instrução em sequência (ou seja, a instrução localizada no próximo endereço de memória mais alto). Assim, por exemplo, considere um computador em que cada instrução ocupa uma palavra de memória de 16 bits. Suponha que o contador de programa esteja definido no local 300. O processador em seguida buscará a instrução no local 300. Nos ciclos de instrução seguintes, ele buscará instruções dos locais 301, 302, 303 e assim por diante. Essa sequência pode ser alterada, como explicamos logo em seguida.

A instrução lida é carregada em um registrador no processador, conhecido como registrador de instrução (IR). A instrução contém bits que especificam a ação que o processador deve tomar. O processador interpreta a instrução e realiza a ação solicitada. Em geral, essas ações estão em uma destas quatro categorias:

- **Processador-memória:** os dados podem ser transferidos do processador para a memória ou da memória para o processador.
- **Processador-E/S:** os dados podem ser transferidos de ou para um dispositivo periférico, transferindo entre o processador e um módulo de E/S.
- **Processamento de dados:** o processador pode realizar alguma operação aritmética ou lógica sobre os dados.

Figura 3.3 Ciclo de instrução básico



- **Controle:** uma instrução pode especificar que a sequência de execução seja alterada. Por exemplo, o processador pode buscar uma instrução do local 149, que especifica que a próxima instrução seja do local 182. O processador se lembrará desse fato definindo o contador de programa como 182. Assim, no próximo ciclo de busca, a instrução será apanhada do local 182, em vez de 150.

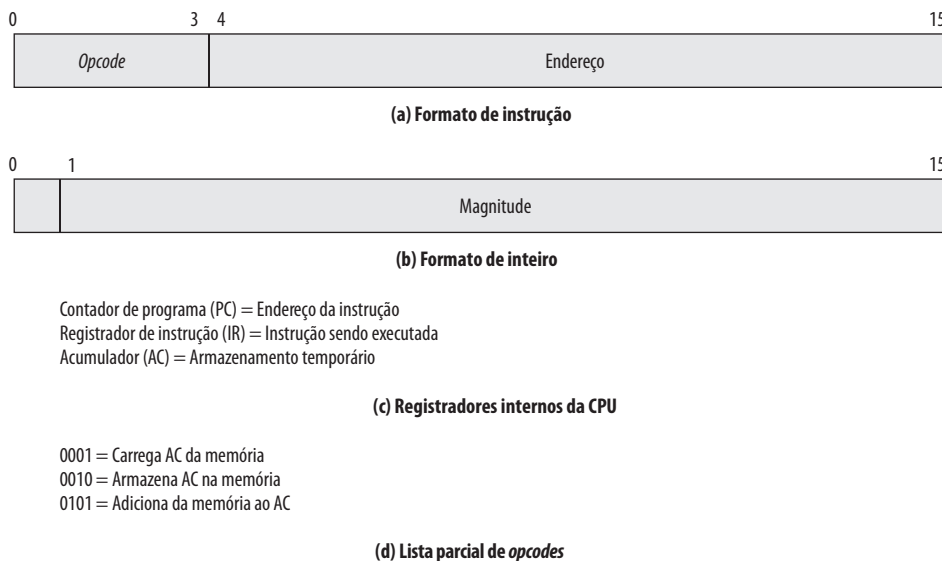
A execução de uma instrução pode envolver uma combinação dessas ações.

Considere um exemplo simples, usando uma máquina hipotética, que inclui as características listadas na Figura 3.4. O processador contém um único registrador de dados, chamado acumulador (AC). Instruções e dados possuem 16 bits de extensão. Assim, é conveniente organizar a memória usando palavras de 16 bits. O formato de instrução oferece 4 bits para o *opcode*, de modo que pode haver até $2^4 = 16$ *opcodes* diferentes, e até $2^{12} = 4096$ (4K) palavras de memória podem ser endereçadas diretamente.

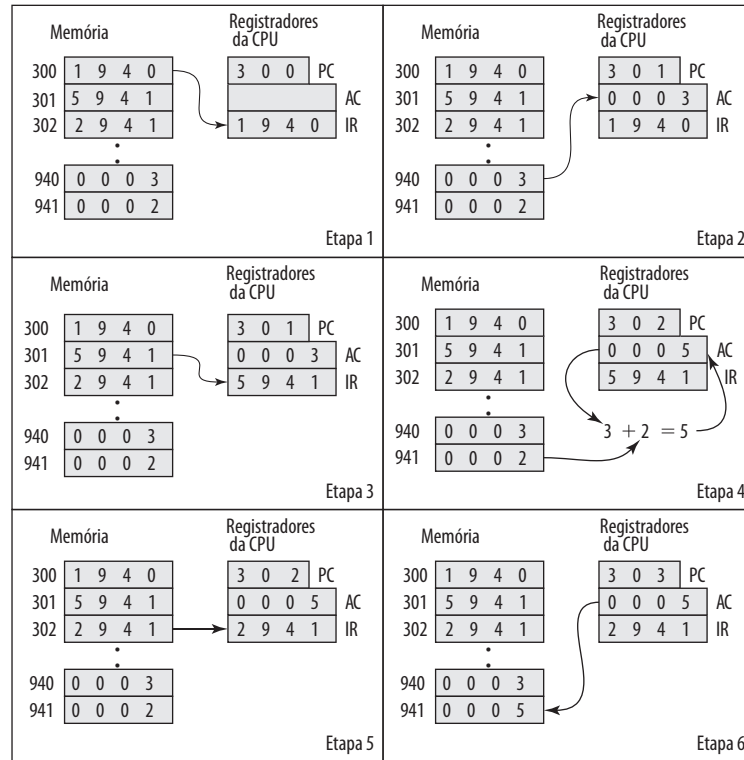
A Figura 3.5 ilustra uma execução parcial de programa, mostrando as partes relevantes dos registradores de memória e processador.¹ O fragmento de programa mostrado soma o conteúdo da palavra de memória no endereço 940 ao conteúdo da palavra de memória no endereço 941 e armazena o resultado no segundo local. Três instruções, que podem ser descritas como três ciclos de busca e três de execução, são necessárias:

1. O PC contém 300, o endereço da primeira instrução. Essa instrução (o valor 1940 em hexadecimal) é carregada no registrador de instrução IR e o PC é incrementado. Observe que esse processo envolve o uso de um registrador de endereço de memória (MAR) e um registrador de buffer de memória (MBR). Para simplificar, esses registradores intermediários são ignorados.
2. Os 4 primeiros bits (primeiro dígito hexadecimal) no IR indicam que o AC deve ser carregado. Os 12 bits restantes (três dígitos hexadecimais) especificam o endereço (940) de onde os dados devem ser carregados.
3. A próxima instrução (5941) é buscada do local 301 e o PC é incrementado.
4. O conteúdo antigo do AC e o conteúdo do local 941 são acrescentados e o resultado é armazenado no AC.
5. A próxima instrução (2941) é buscada do local 302 e o PC é incrementado.
6. O conteúdo do AC é armazenado no local 941.

Figura 3.4 Características de uma máquina hipotética



¹ É usada a notação hexadecimal, na qual cada dígito representa 4 bits. Essa é a notação mais conveniente para representar o conteúdo da memória e registradores quando o tamanho da palavra é um múltiplo de 4. Veja, no Capítulo 19, uma revisão básica sobre sistemas numéricos (decimal, binário, hexadecimal).

Figura 3.5 Exemplo de execução de programa (conteúdo da memória e dos registradores em hexadecimal)

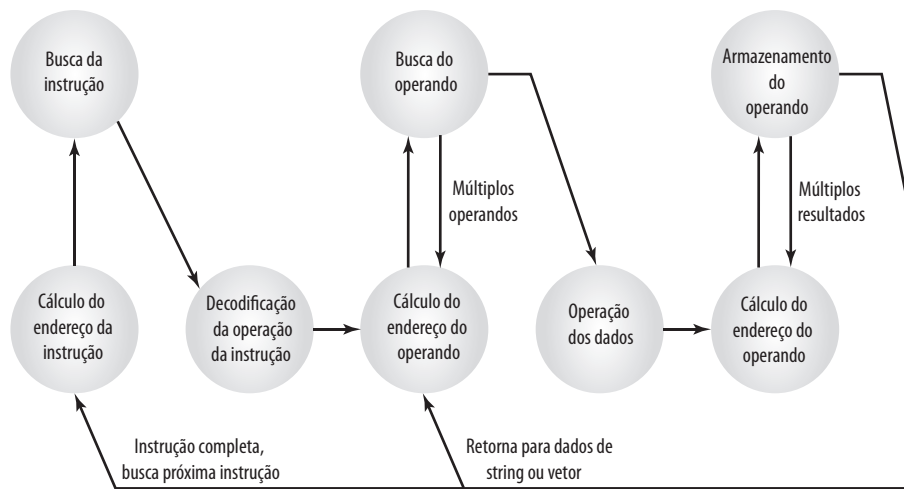
Neste exemplo, três ciclos de instrução, cada um consistindo em um ciclo de busca e um ciclo de execução, são necessários para somar o conteúdo do local 940 ao conteúdo de 941. Com um conjunto de instruções mais complexo, menos ciclos seriam necessários. Alguns processadores mais antigos, por exemplo, incluíam instruções contendo mais de um endereço de memória. Assim, o ciclo de execução para determinada instrução em tais processadores poderia envolver mais de uma referência à memória. Além disso, em vez de referências à memória, uma instrução pode especificar uma operação de E/S.

Por exemplo, o processador PDP-11 inclui uma instrução, expressa simbolicamente como ADD B,A, que armazena a soma do conteúdo dos locais de memória B e A ao local de memória A. Ocorre um único ciclo de instrução com as seguintes etapas:

- Buscar a instrução ADD.
- Ler o conteúdo do local de memória A no processador.
- Ler o conteúdo do local de memória B para o processador. Para que o conteúdo de A não seja perdido, o processador precisa ter pelo menos dois registradores para armazenar valores de memória, ao invés de um único acumulador.
- Somar os dois valores.
- Escrever o resultado do processador no local de memória A.

Assim, o ciclo de execução para determinada instrução pode envolver mais de uma referência à memória. Além disso, em vez de referências à memória, uma instrução pode especificar uma operação de E/S. Lembrando dessas considerações adicionais, a Figura 3.6 oferece uma visão mais detalhada do ciclo de instrução básico da Figura 3.3. A figura está na forma de um diagrama de estado. Para qualquer ciclo de instrução dado, alguns estados podem ser nulos e outros podem ser visitados mais de uma vez. Os estados podem ser descritos da seguinte forma:

- **Cálculo de endereço de instrução (iac, do inglês *instruction address calculation*):** determina o endereço da próxima instrução a ser executada. Normalmente, isso envolve acrescentar um número fixo ao

Figura 3.6 Diagrama de estado do ciclo de instrução

endereço da instrução anterior. Por exemplo, se cada instrução tem 16 bits de extensão e a memória é organizada em palavras de 16 bits, então some 1 ao endereço anterior. Se, ao invés disso, a memória é organizada como bytes de 8 bits endereçáveis individualmente, então some 2 ao endereço anterior.

- **Busca da instrução (if, do inglês *instruction fetch*):** lê a instrução do seu local da memória para o processador.
- **Decodificação da operação da instrução (iod, do inglês *instruction operation decoding*):** analisa a instrução para determinar o tipo de operação a ser realizado e o operando ou operandos a serem utilizados.
- **Cálculo do endereço do operando (oac, do inglês *operation address calculation*):** se a operação envolve referência a um operando na memória ou disponível via E/S, então determina o endereço do operando.
- **Busca do operando (of, do inglês *operation fetch*):** busca o operando da memória ou o lê da E/S.
- **Operação dos dados (do, do inglês *data operation*):** realiza a operação indicada na instrução.
- **Armazenamento do operando (os, do inglês *operand store*):** escreve o resultado na memória ou envia para a E/S.

Os estados na parte superior da Figura 3.6 envolvem uma troca entre o processador e a memória ou um módulo de E/S. Os estados na parte inferior do diagrama envolvem apenas operações internas do processador. O estado oac aparece duas vezes, pois uma instrução pode envolver uma leitura, uma escrita ou ambos. Porém, a ação realizada durante esse estado é fundamentalmente a mesma nos dois casos, e, por isso, apenas um único identificador de estado é necessário.

Observe também que o diagrama possibilita múltiplos operandos e resultados, pois algumas instruções em algumas máquinas exigem isso. Por exemplo, a instrução ADD A,B do PDP-11 resulta na seguinte sequência de estados: iac, if, iod, oac, of, oac, of, do, oac, os.

Finalmente, em algumas máquinas, uma única instrução pode especificar uma operação a ser realizada sobre um vetor (array unidimensional) de números ou uma string (array unidimensional) de caracteres. Como a Figura 3.6 indica, isso envolveria operações repetitivas de busca e/ou armazenamento de operando.



Interrupções

Praticamente todos os computadores oferecem um mecanismo por meio do qual outros módulos (E/S, memória) podem interromper o processamento normal do processador. A Tabela 3.1 lista as classes de interrupção mais comuns. A natureza específica dessas interrupções será examinada mais adiante neste livro, especialmente nos capítulos 7 e 12. Porém, precisamos introduzir o conceito agora, para entender mais claramente a natureza do ciclo de instrução e as implicações das interrupções sobre a estrutura de interconexão. O leitor não precisa se preocupar neste estágio com os detalhes da geração e processamento de interrupções, mas apenas se concentrar na comunicação entre os módulos, resultante das interrupções.

Tabela 3.1 Classes de interrupções

Programa	Gerada por alguma condição que ocorre como resultado da execução de uma instrução, como o <i>overflow</i> aritmético, divisão por zero, tentativa de executar uma instrução de máquina ilegal ou referência fora do espaço de memória permitido para o usuário.
Timer	Gerada por um timer dentro do processo. Isso permite que o sistema operacional realize certas funções regularmente.
E/S	Gerada por um controlador de E/S para sinalizar o término normal de uma operação ou para sinalizar uma série de condições de erro.
Falha de hardware	Gerada por uma falha como falta de energia ou erro de paridade de memória.

As interrupções são fornecidas primeiramente como um modo de melhorar a eficiência do processamento. Por exemplo, a maioria dos dispositivos externos é muito mais lenta do que o processador. Suponha que o processador esteja transferindo dados a uma impressora usando o esquema de ciclo de instrução da Figura 3.3. Após cada operação de escrita, o processador deve parar e permanecer ocioso até que a impressora o alcance. A extensão dessa pausa pode estar na ordem de muitas centenas ou mesmo milhares de ciclos de instrução que não envolvem memória. Claramente, esse é um grande desperdício de uso do processador.

A Figura 3.7a ilustra esse estado de coisas. O programa do usuário realiza uma série de chamadas WRITE intercaladas com processamento. Os segmentos de código 1, 2 e 3 referem-se às sequências de instruções que não envolvem E/S. As chamadas WRITE são para um programa de E/S que é um utilitário do sistema e que realizará a operação de E/S real. O programa de E/S consiste em três seções:

- Uma sequência de instruções, rotuladas como 4 na figura, para preparar para a operação de E/S real. Isso pode incluir a cópia dos dados para a saída em um buffer especial e a preparação dos parâmetros para um comando de dispositivo.
- O comando de E/S real. Sem o uso de interrupções, quando esse comando é emitido, o programa precisa esperar pelo dispositivo de E/S para realizar a função solicitada (ou sondar o dispositivo periodicamente). O programa poderia esperar simplesmente realizando uma operação de teste repetidamente, para determinar se a operação de E/S terminou.
- Uma sequência de instruções, rotulada como 5 na figura, para completar a operação. Isso pode incluir a marcação de um flag, indicando o sucesso ou a falha da operação.

Como a operação de E/S pode levar um tempo relativamente longo para terminar, o programa de E/S fica preso, esperando que a operação termine; daí o programa de E/S ser interrompido no ponto da chamada WRITE por algum período considerável.

INTERRUPÇÕES E O CICLO DE INSTRUÇÃO Com as interrupções, o processador pode estar engajado na execução de outras instruções enquanto uma operação de E/S está em andamento. Considere o fluxo de controle na Figura 3.7b. Como antes, o programa do usuário alcança um ponto em que faz uma chamada do sistema na forma de uma chamada WRITE. O programa de E/S que é invocado, nesse caso, consiste apenas no código de preparação e o comando de E/S real. Depois que essas poucas instruções tiverem sido executadas, o controle retorna ao programa do usuário. Enquanto isso, o dispositivo externo está ocupado aceitando e imprimindo dados vindos da memória do computador. Essa operação de E/S é realizada simultaneamente com a execução de instruções no programa do usuário.

Quando o dispositivo externo estiver pronto para ser atendido — ou seja, quando estiver pronto para aceitar mais dados do processador —, o módulo de E/S para o dispositivo externo envia um sinal de *requisição de interrupção* ao processador. O processador responde suspendendo a operação do programa atual, desviando para um programa para atender a esse dispositivo de E/S em particular, conhecido como tratador de interrupção, e retomando a execução original depois que o dispositivo for atendido. Os pontos em que essas interrupções ocorrem são indicados por um asterisco na Figura 3.7b.

Do ponto de vista do programa do usuário, uma interrupção é apenas isso: uma interrupção da sequência de execução normal. Quando o processamento da interrupção tiver terminado, a execução retoma (Figura 3.8).

Figura 3.7 Fluxo de controle do programa sem e com interrupções

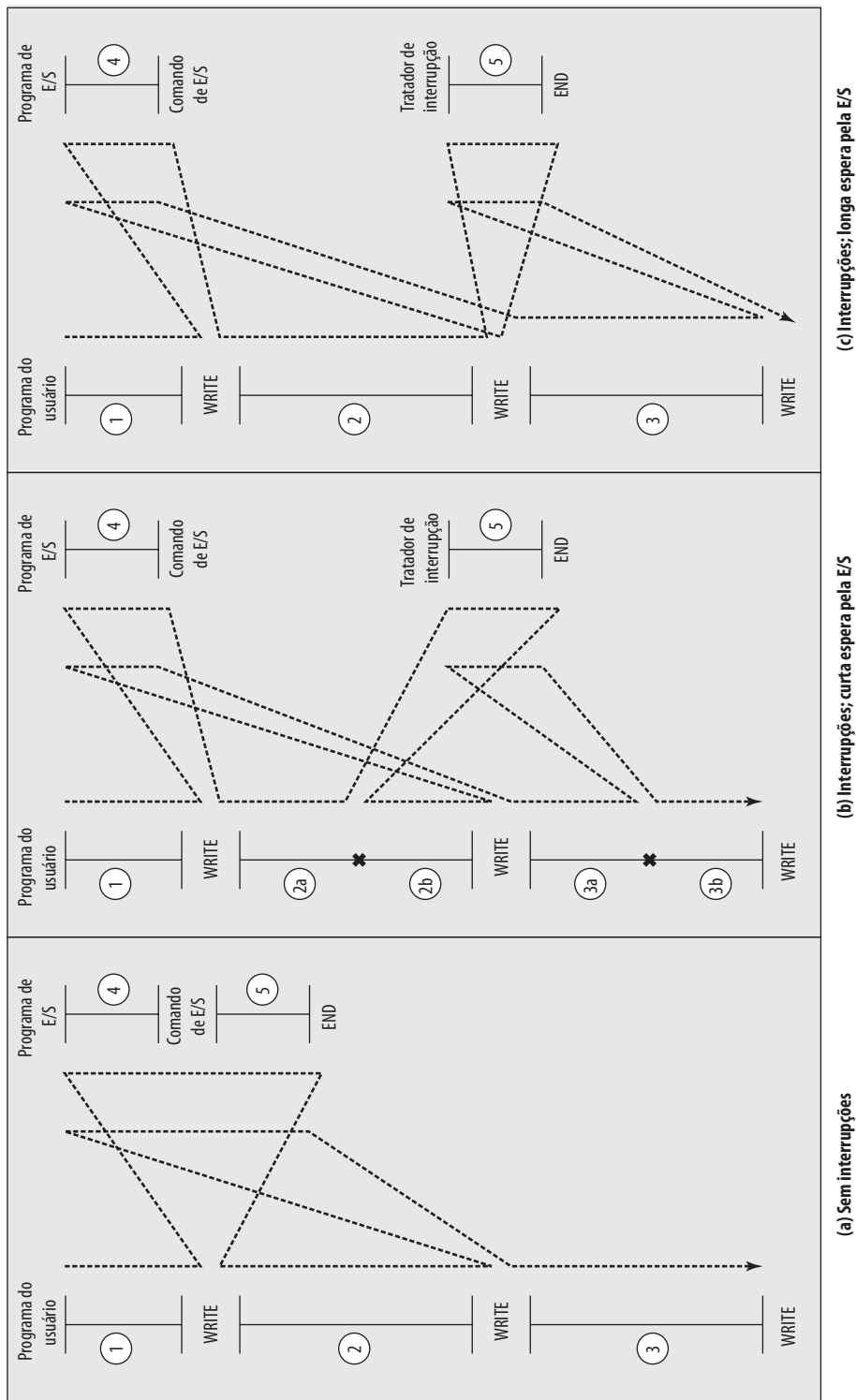
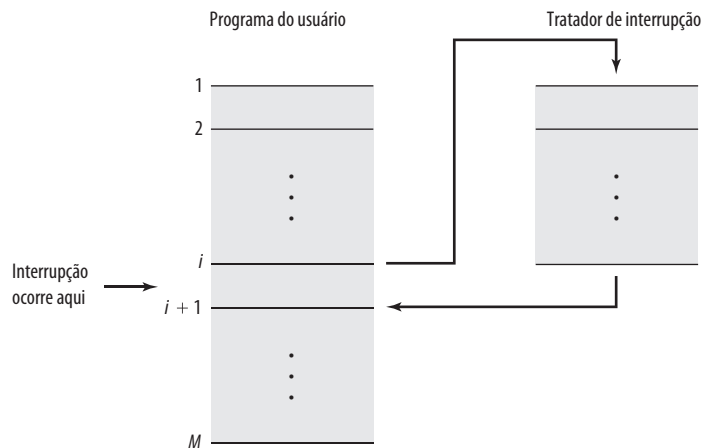


Figura 3.8 Transferência de controle via interrupções

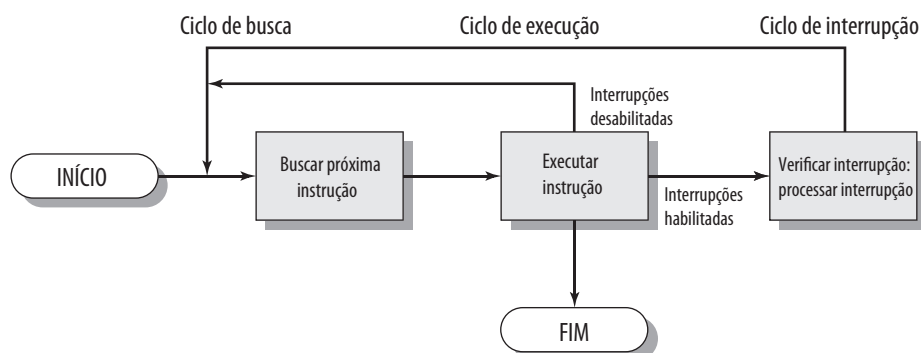
Assim, o programa do usuário não precisa conter qualquer código especial para acomodar as interrupções; o processador e o sistema operacional são responsáveis por suspender o programa do usuário e depois retomá-lo no mesmo ponto.

Para acomodar as interrupções, um *ciclo de interrupção* é acrescentado ao ciclo de instrução, como mostra a Figura 3.9. No ciclo de interrupção, o processador verifica se houve alguma interrupção, que é indicada pela presença de um sinal de interrupção. Se nenhuma interrupção estiver pendente, o processador prossegue para o ciclo de busca, lendo a próxima instrução do programa atual. Se uma interrupção estiver pendente, o processador faz o seguinte:

- Suspende a execução do programa que está sendo executado e salva seu contexto. Isso significa salvar o endereço da próxima instrução a ser executada (conteúdo atual do contador de programa) e quaisquer outros dados relevantes à atividade atual do processador.
- Armazena no contador do programa o endereço inicial de uma rotina de *tratamento de interrupção*.

O processador, agora, continua com o ciclo de busca, obtendo a primeira instrução da rotina de tratamento de interrupção, que tratará a interrupção. O programa tratador de interrupção geralmente faz parte do sistema operacional. Normalmente, esse programa determina a natureza da interrupção e realiza quaisquer ações necessárias. No exemplo que usamos, o tratador determina qual módulo de E/S gerou a interrupção e pode se desviar para um programa que escreverá mais dados nesse módulo de E/S. Quando a rotina de tratamento de interrupção terminar, o processador pode retomar a execução do programa do usuário no ponto da interrupção.

É evidente que existe algum *overhead* envolvido nesse processo. Instruções extras precisam ser executadas (no tratador de interrupção) para determinar a natureza da interrupção e decidir sobre a ação apropriada. Apesar disso,

Figura 3.9 Ciclo de instrução com interrupções

devido à quantidade de tempo relativamente grande que seria desperdiçada pela simples espera por uma operação de E/S, o processador pode ser empregado de modo muito mais eficiente com o uso de interrupções.

Para apreciar o ganho na eficiência, considere a Figura 3.10, que é um diagrama de tempo baseado no fluxo de controle nas figuras 3.7a e 3.7b. As figuras 3.7b e 3.10 consideram que o tempo exigido para a operação de E/S é relativamente curto: menos do que o tempo para completar a execução das instruções entre as operações de escrita no programa do usuário. O caso mais típico, especialmente para um dispositivo lento como uma impressora, é que a operação de E/S levará muito mais tempo do que a execução de uma sequência de instruções do usuário. A Figura 3.7c indica esse estado de coisas. Nesse caso, o programa do usuário alcança a segunda chamada WRITE antes que a operação de E/S gerada pela primeira chamada termine. O resultado é que o programa do usuário está travado nesse ponto. Quando a operação de E/S anterior terminar, essa nova chamada WRITE poderá ser processada, e uma nova operação de E/S poderá ser iniciada. A Figura 3.11 mostra a sincronização para essa situação com e sem o uso de interrupções. Podemos ver que ainda existe um ganho na eficiência, pois parte do tempo durante o qual a operação de E/S está sendo realizada sobrepõe a execução das instruções do usuário.

A Figura 3.12 mostra um diagrama de estado do ciclo de instruções revisado, que inclui o processamento do ciclo de interrupção.

INTERRUPÇÕES MÚLTIPLAS A discussão até aqui focou apenas a ocorrência de uma única interrupção. Suponha, porém, que ocorram múltiplas interrupções. Por exemplo, um programa pode estar recebendo dados de uma linha de comunicações e imprimindo resultados. A impressora gerará uma interrupção toda vez que completar uma operação de impressão. O controlador da linha de comunicação gerará uma interrupção toda vez que uma unidade de dados chegar. A unidade poderia ser um único caractere ou um bloco, dependendo da natureza do controle das comunicações. De qualquer forma, é possível que uma interrupção de comunicações ocorra enquanto uma interrupção de impressora esteja sendo processada.

Duas técnicas podem ser utilizadas para lidar com múltiplas interrupções. A primeira é desativar as interrupções enquanto uma interrupção estiver sendo processada. Uma *interrupção desabilitada* significa simplesmente que o

Figura 3.10 Sincronização do programa: espera curta pela E/S

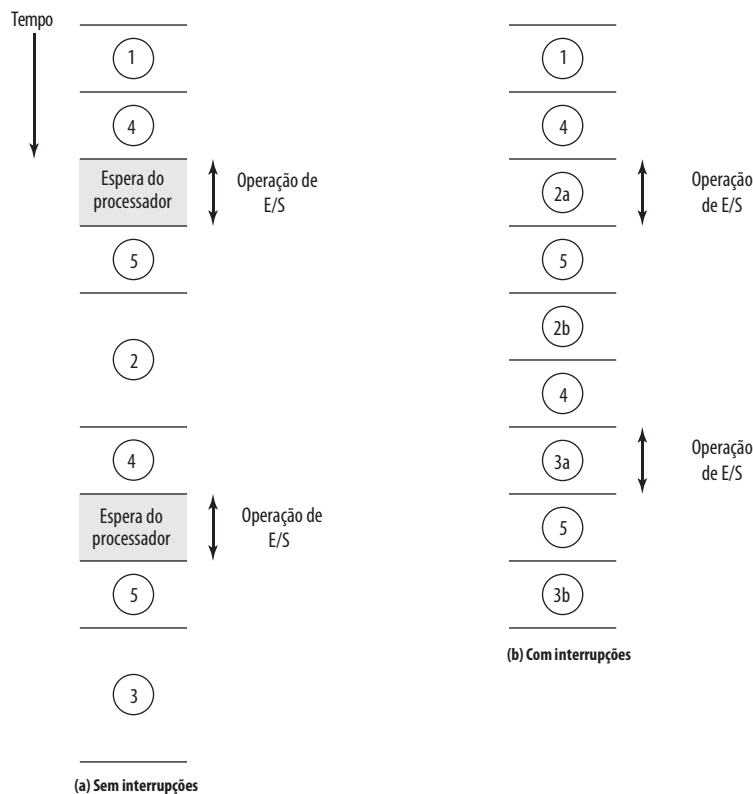
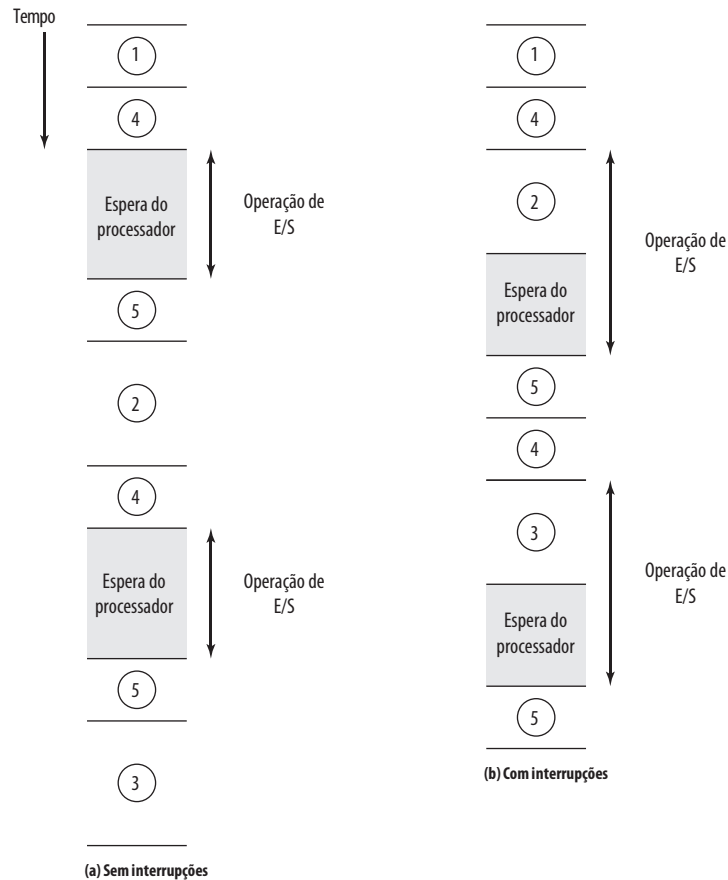
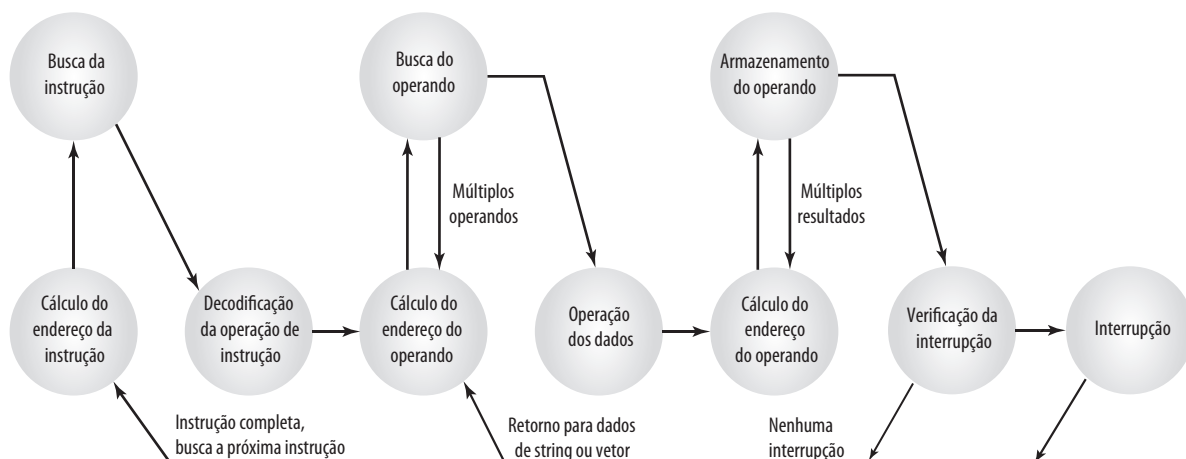


Figura 3.11 Sincronização do programa: espera longa pela E/S

processador pode ignorar e ignorará esse sinal de requisição de interrupção. Se uma interrupção ocorrer durante esse tempo, ela geralmente permanece pendente e será verificada pelo processador depois que ele tiver habilitado as interrupções. Assim, quando um programa do usuário estiver sendo executado e houver uma interrupção, as interrupções são imediatamente desabilitadas. Depois que a rotina de tratamento de interrupção terminar, as interrupções

Figura 3.12 Diagrama de estado do ciclo de instruções, com interrupções

são habilitadas antes que o programa do usuário retome, e o processador verifica se houve interrupções adicionais. Essa técnica é boa e simples, pois as interrupções são tratadas em ordem estritamente sequencial (Figura 3.13a).

A desvantagem da técnica anterior é que ela não leva em consideração a prioridade relativa ou necessidades de tempo crítico. Por exemplo, quando a entrada chega da linha de comunicações, ela pode precisar ser absorvida rapidamente, para dar espaço para mais entrada. Se o primeiro lote de entrada não for processado antes que o segundo lote chegue, dados poderão ser perdidos.

Uma segunda técnica é definir prioridades para interrupções e permitir que uma interrupção de maior prioridade faça com que um tratamento de interrupção com menor prioridade seja interrompido (Figura 3.13b). Como um exemplo dessa segunda técnica, considere um sistema com três dispositivos de E/S: uma impressora, um disco e uma linha de comunicações, com prioridades cada vez maiores de 2, 4 e 5, respectivamente. A Figura 3.14, baseada em um exemplo de Tanenbaum e Woodhull (1997^a), ilustra uma sequência possível. Um usuário inicia em $t = 0$. Em $t = 10$, ocorre uma interrupção da impressora; a informação do usuário é colocada na pilha do sistema e a execução continua na rotina de serviço de interrupção (ISR, do inglês *interrupt service routine*). Enquanto essa rotina ainda está sendo executada, em $t = 15$, ocorre uma interrupção de comunicação. Como a linha de comunicações tem prioridade mais alta que a impressora, a interrupção é considerada. A ISR da impressora é interrompida, seu estado é colocado na pilha e a execução continua na ISR de comunicação. Enquanto essa rotina está sendo executada, ocorre uma interrupção de disco ($t = 20$). Como essa interrupção tem prioridade menor, ela é simplesmente retida, e a ISR de comunicação é executada até o final.

Figura 3.13 Transferência de controle com múltiplas interrupções

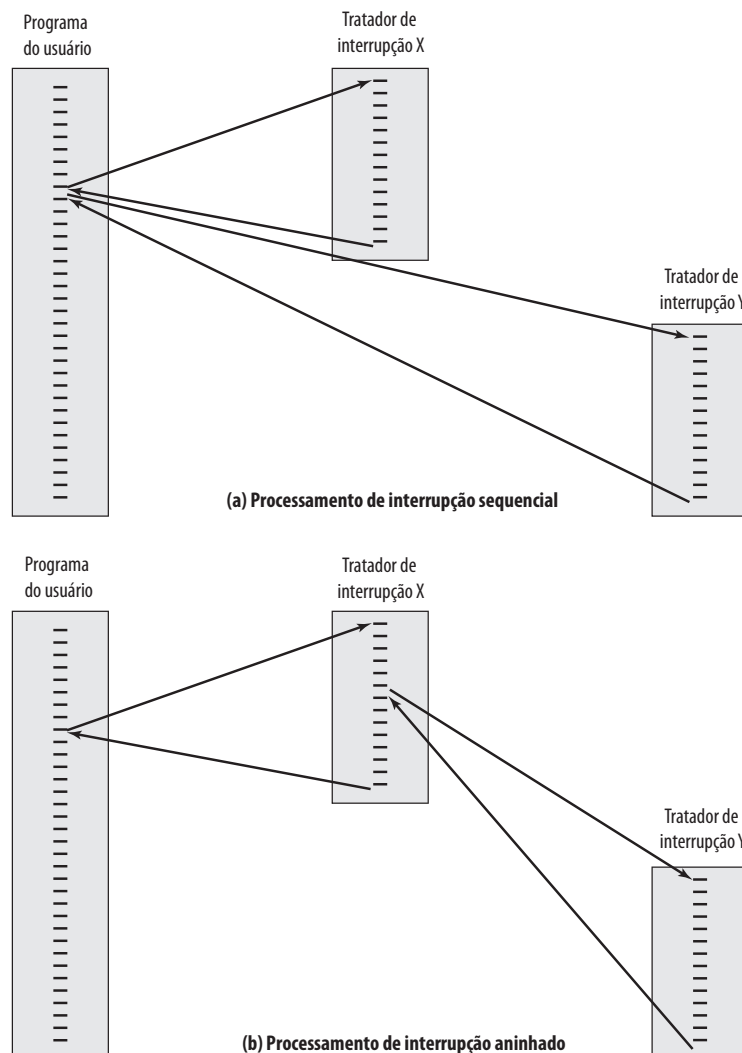
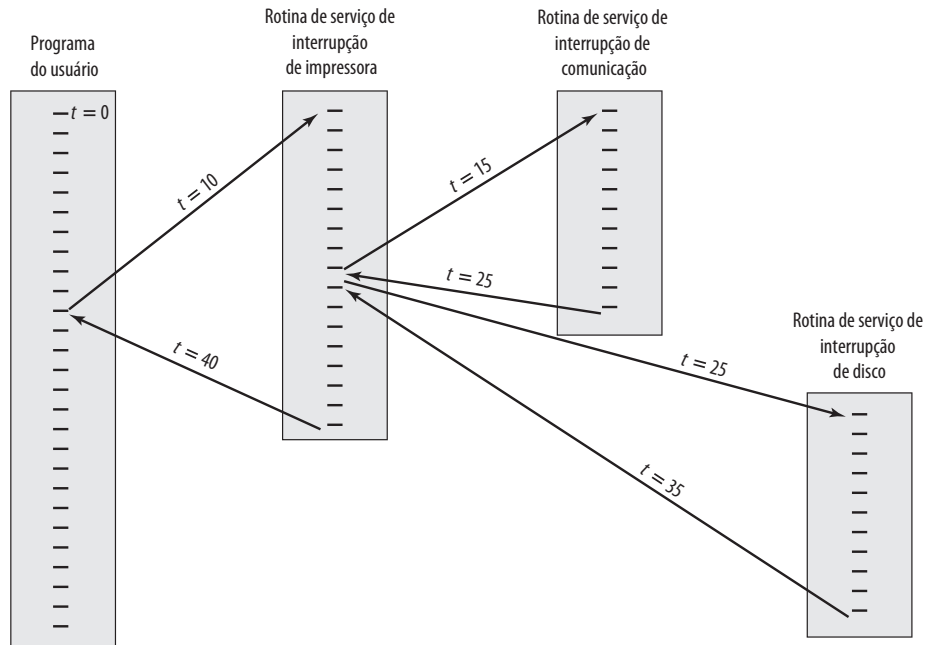


Figura 3.14 Exemplo de sequência de tempo de múltiplas interrupções

Quando a ISR de comunicação termina ($t = 25$), o estado anterior do processador, que é a execução da ISR de impressora, é restaurado. Porém, antes mesmo que uma única instrução nessa rotina possa ser executada, o processador aceita a interrupção de disco de maior prioridade e transfere o controle para a ISR de disco. Somente quando essa rotina terminar ($t = 35$) é que a ISR de impressora é retomada. Quando a última rotina termina ($t = 40$), o controle finalmente retorna ao programa do usuário.



Função de E/S

Até aqui, discutimos a operação do computador de acordo com o controle do processador, e vimos principalmente a interação entre processador e memória. A discussão apenas aludiu ao papel do componente de E/S. Esse papel é discutido com detalhes no Capítulo 7, mas apresentamos um rápido resumo aqui.

Um módulo de E/S (por exemplo, um controlador de disco) pode trocar dados diretamente com o processador. Assim como o processador pode iniciar uma leitura ou escrita com a memória, designando o endereço de um local específico, o processador também pode ler ou escrever dados em um módulo de E/S. Nesse último caso, o processador identifica um dispositivo específico que é controlado por um módulo de E/S em particular. Assim, poderia ocorrer uma sequência de instruções semelhante em formato à da Figura 3.5, com instruções de E/S em vez de instruções de referência à memória.

Em alguns casos, é desejável permitir que as trocas de E/S ocorram diretamente com a memória. Nesse caso, o processador concede a um módulo de E/S a autoridade de ler ou escrever na memória, de modo que a transferência entre E/S e memória pode ocorrer sem prender o processador. Durante essa transferência, o módulo de E/S emite comandos de leitura ou escrita à memória, tirando do processador a responsabilidade pela troca. Essa operação é conhecida como acesso direto à memória (DMA, do inglês *direct memory access*), e é examinada no Capítulo 7.



3.3 Estrutura de interconexão

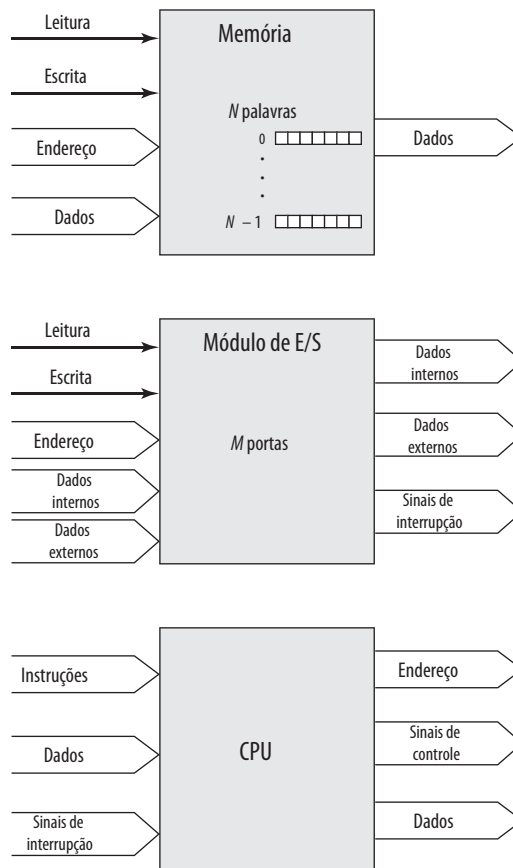
Um computador consiste em um conjunto de componentes ou módulos de três tipos básicos (processador, memória e E/S) que se comunicam entre si. Com efeito, um computador é uma rede de módulos básicos. Assim, é preciso haver caminhos para a conexão dos módulos.

A coleção de caminhos conectando os diversos módulos é chamada de *estrutura de interconexão*. O projeto dessa estrutura depende das trocas que precisam ser feitas entre os módulos.

A Figura 3.15 sugere os tipos de trocas que são necessárias, indicando as principais formas de entrada e saída para cada tipo de módulo.²

- **Memória:** normalmente, um módulo de memória consiste em N palavras de mesmo tamanho. Cada palavra recebe um endereço numérico exclusivo (0, 1, ..., $N - 1$). Uma palavra de dados pode ser lida ou escrita na memória. A natureza da operação é indicada por sinais de controle de leitura e escrita. O local para a operação é especificado por um endereço.
- **Módulo de E/S:** por um ponto de vista interno (ao sistema de computação), a E/S é funcionalmente semelhante à memória. Existem duas operações, leitura e escrita. Além disso, um módulo de E/S pode controlar mais de um dispositivo externo. Podemos nos referir a cada uma das interfaces para um dispositivo externo

Figura 3.15 Módulos do computador



² As setas grossas representam múltiplas linhas de sinal transportando múltiplos bits de informação em paralelo. Cada seta estreita representa uma única linha de sinal.

como uma *porta*, dando a cada uma um endereço exclusivo (por exemplo, 0, 1, ..., $M - 1$). Além disso, existem caminhos de dados externos para a entrada e saída de dados com um dispositivo externo. Finalmente, um módulo de E/S pode ser capaz de enviar sinais de interrupção ao processador.

- **Processador:** o processador lê instruções e dados, escreve dados após o processamento e usa sinais de controle para controlar a operação geral do sistema. Ele também recebe sinais de interrupção.

A lista anterior define os dados a serem trocados. A estrutura de interconexão deve admitir os seguintes tipos de transferências:

- **Memória para processador:** o processador lê uma instrução ou uma unidade de dados da memória.
- **Processador para memória:** o processador escreve uma unidade de dados na memória.
- **E/S para processador:** o processador lê dados de um dispositivo de E/S por meio de um módulo de E/S.
- **Processador para E/S:** o processador envia dados para o dispositivo de E/S.
- **E/S de ou para a memória:** para esses dois casos, um módulo de E/S tem permissão para trocar dados diretamente com a memória, sem passar pelo processador, usando o DMA.

Com o passar dos anos, diversas estruturas de interconexão foram experimentadas. De longe, a mais comum é o barramento e diversas estruturas de barramento múltiplo. O restante deste capítulo é dedicado a uma avaliação das estruturas de barramento.



3.4 Interconexão de barramento

Um barramento é um caminho de comunicação que conecta dois ou mais dispositivos. Uma característica-chave de um barramento é que ele é um meio de transmissão compartilhado. Múltiplos dispositivos se conectam ao barramento, e um sinal transmitido por qualquer dispositivo está disponível para recepção por todos os outros dispositivos conectados ao barramento. Se dois dispositivos transmitirem durante o mesmo período, seus sinais serão sobrepostos e ficarão distorcidos. Assim, somente um dispositivo de cada vez pode transmitir com sucesso.

Tipicamente, um barramento consiste em múltiplos caminhos de comunicação, ou linhas. Cada linha é capaz de transmitir sinais representando o binário 1 e o binário 0. Com o tempo, uma sequência de dígitos binários pode ser transmitida por uma única linha. Juntas, várias linhas de um barramento podem ser usadas para transmitir dígitos binários simultaneamente (em paralelo). Por exemplo, uma unidade de dados de 8 bits pode ser transmitida por oito linhas de barramento.

Os sistemas de computação contêm diversos barramentos diferentes, que oferecem caminhos entre os componentes em diversos níveis da hierarquia do sistema de computação. Um barramento que conecta os principais componentes do computador (processador, memória, E/S) é chamado de *barramento do sistema*. As estruturas de interconexão de computador mais comuns são baseadas no uso de um ou mais barramentos do sistema.

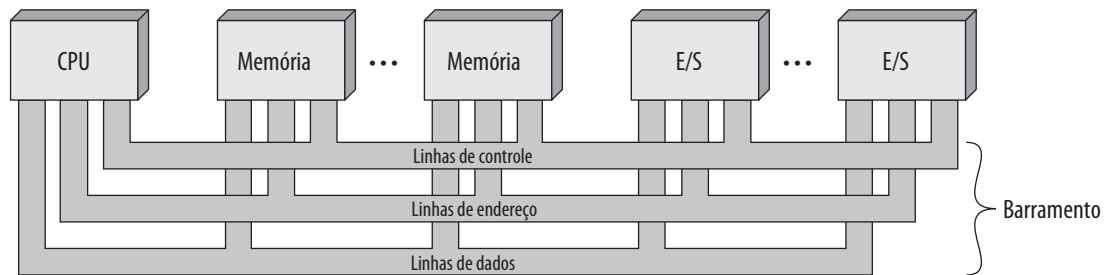


Estrutura de barramento

Um barramento do sistema consiste, normalmente, em cerca de 50 a centenas de linhas separadas. Cada linha recebe um significado ou função em particular. Embora existam muitos projetos de barramento diferentes, em qualquer barramento as linhas podem ser classificadas em três grupos funcionais (Figura 3.16): linhas de dados, endereço e controle. Além disso, pode haver linhas de distribuição de potência, que fornecem energia aos módulos conectados.

As **linhas de dados** oferecem um caminho para movimentação de dados entre os módulos do sistema. Essas linhas, coletivamente, são chamadas de *barramento de dados*. O barramento de dados pode consistir em 32, 64, 128 ou ainda mais linhas separadas, sendo que o número de linhas é conhecido como a *largura* do barramento de dados. Como cada linha só pode transportar 1 bit de cada vez, o número de linhas determina quantos bits podem ser transferidos de uma só vez. A largura do barramento de dados é um fator chave para determinar o desempenho geral do sistema. Por exemplo, se o barramento de dados tiver 32 bits de largura e cada instrução tiver 64 bits de extensão, então o processador precisa acessar o módulo de memória duas vezes durante cada ciclo de instrução.

As **linhas de endereço** são usadas para designar a origem ou o destino dos dados no barramento de dados. Por exemplo, se o processador deseja ler uma palavra (8, 16 ou 32 bits) de dado da memória, ele coloca

Figura 3.16 Esquema de interconexão de barramento

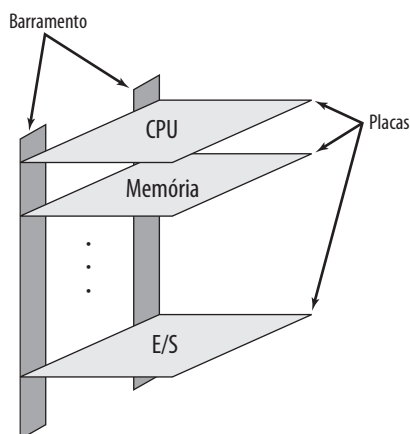
o endereço da palavra desejada nas linhas de endereço. Claramente, a largura do barramento de endereço determina a capacidade de memória máxima possível do sistema. Além do mais, as linhas de endereço geralmente também são usadas para endereçar portas de E/S. Normalmente, os bits de ordem mais alta são usados para selecionar um módulo em particular no barramento, e os bits de ordem mais baixa selecionam um local de memória ou porta de E/S dentro do módulo. Por exemplo, em um barramento de endereço de 8 bits, o endereço 01111111 e mais baixos poderiam referenciar locais em um módulo de memória (módulo 0) com 128 palavras de memória, e o endereço 10000000 e mais altos poderiam referenciar dispositivos conectados a um módulo de E/S (módulo 1).

As **linhas de controle** são usadas para controlar o acesso e o uso das linhas de dados e endereço. Como as linhas de dados e endereço são compartilhadas por todos os componentes, é preciso haver um meio de controlar seu uso. Os sinais de controle transmitem informações de comando e sincronização entre os módulos do sistema. Os sinais de sincronização indicam a validade da informação de dados e endereço. Os sinais de comando especificam operações a serem realizadas. As linhas de controle típicas incluem:

- **Escrita de memória:** faz com que os dados no barramento sejam escritos no local endereçado.
- **Leitura de memória:** faz com que os dados do local endereçado sejam colocados no barramento.
- **Escrita de E/S:** faz com que os dados no barramento sejam enviados para a porta de E/S endereçada.
- **Leitura de E/S:** faz com que os dados da porta de E/S endereçada sejam colocados no barramento.
- **ACK de transferência:** indica que dados foram aceitos do barramento ou colocados nele.
- **Solicitação de barramento (bus request):** indica que um módulo precisa obter controle do barramento.
- **Concessão de barramento (bus grant):** indica que um módulo solicitante recebeu controle do barramento.
- **Requisição de interrupção (interrupt request):** indica que a interrupção está pendente.
- **ACK de interrupção:** confirma que a interrupção pendente foi reconhecida.
- **Clock:** é usado para operações de sincronização.
- **Reset:** inicializa todos os módulos.

A operação do barramento é a seguinte. Se um módulo deseja enviar dados para outro, ele precisa fazer duas coisas: (1) obter o uso do barramento e (2) transferir dados por meio do barramento. Se um módulo quiser requisitar dados de outro módulo, ele precisa (1) obter o uso do barramento e (2) transferir uma requisição ao outro módulo pelas linhas de controle e endereço apropriadas. Depois, ele precisa esperar que esse segundo módulo envie os dados.

Fisicamente, o barramento do sistema é, na realidade, uma série de condutores elétricos paralelos. No arranjo clássico do barramento, esses condutores são linhas de metal coladas a uma placa de circuito impresso. O barramento se estende por todos os componentes do sistema, cada um preso a algumas ou todas as linhas do barramento. O arranjo físico clássico é representado na Figura 3.17. Neste exemplo, o barramento consiste em duas colunas verticais de condutores. Em intervalos regulares ao longo das colunas, existem pontos de conexão na forma de *slots* que se estendem horizontalmente para dar suporte a uma placa de circuito impresso. Cada um dos principais componentes do sistema ocupa uma ou mais placas e se encaixa no barramento nesses *slots*. O arranjo inteiro está incluído em um chassi. Esse esquema ainda pode ser usado para alguns dos barramentos associados a um sistema de computação. Porém, os sistemas modernos tendem a ter todos os principais componentes na mesma placa com mais elementos no mesmo chip do processador. Assim, um barramento no chip pode conectar

Figura 3.17 Realização física típica de uma arquitetura de barramento

o processador e a memória cache, enquanto um barramento na placa pode conectar o processador à memória principal e a outros componentes.

Esse arranjo é o mais conveniente. Um pequeno sistema de computação pode ser adquirido e depois expandido mais tarde (mais memória, mais E/S) acrescentando-se mais placas. Se um componente em uma placa falhar, essa placa pode facilmente ser removida e substituída.



Hierarquia de barramento múltiplo

Se muitos dispositivos estiverem conectados ao barramento, o desempenho será prejudicado. Existem duas causas principais:

1. Em geral, quanto mais dispositivos conectados ao barramento, maior o tamanho do barramento e, portanto, maior o atraso de propagação. Esse atraso determina o tempo gasto para os dispositivos coordenarem o uso do barramento. Quando o controle do barramento passa de um dispositivo para outro com frequência, esses atrasos de propagação podem afetar visivelmente o desempenho.
2. O barramento pode se tornar um gargalo à medida que a demanda de transferência de dados agregada se aproxima da capacidade do barramento. Esse problema pode ser combatido, até certo ponto, aumentando a taxa de dados que o barramento pode transportar e usando barramentos mais largos (por exemplo, aumentando o barramento de dados de 32 para 64 bits). Porém, como as taxas de dados geradas pelos dispositivos conectados (por exemplo, controladores de gráficos e vídeo, interfaces de rede) estão crescendo rapidamente, essa é uma corrida que um barramento único por fim está destinado a perder.

Assim, a maioria dos sistemas de computação utiliza múltiplos barramentos, geralmente dispostos em uma hierarquia. Uma estrutura tradicional típica aparece na Figura 3.18a. Existe um barramento local que conecta o processador a uma memória cache e que pode aceitar um ou mais dispositivos locais. O controlador da memória cache conecta a cache não apenas a esse barramento local, mas a um barramento do sistema ao qual estão conectados todos os módulos da memória principal. Conforme veremos no Capítulo 4, o uso de uma estrutura de cache isola o processador de um requisito para acessar a memória principal com frequência. Logo, a memória principal pode ser movida do barramento local para o barramento do sistema. Desse modo, as transferências de E/S de e para a memória principal pelo barramento do sistema não interferem com a atividade do processador.

É possível conectar controladores de E/S diretamente no barramento do sistema. Uma solução mais eficiente é utilizar um ou mais barramentos de expansão para essa finalidade. Uma interface de barramento de expansão coloca em um buffer as transferências de dados entre o barramento do sistema e os controladores de E/S no barramento de expansão. Esse arranjo permite que o sistema aceite uma grande variedade de dispositivos de E/S e, ao mesmo tempo, isole o tráfego memória-para-processador do tráfego de E/S.

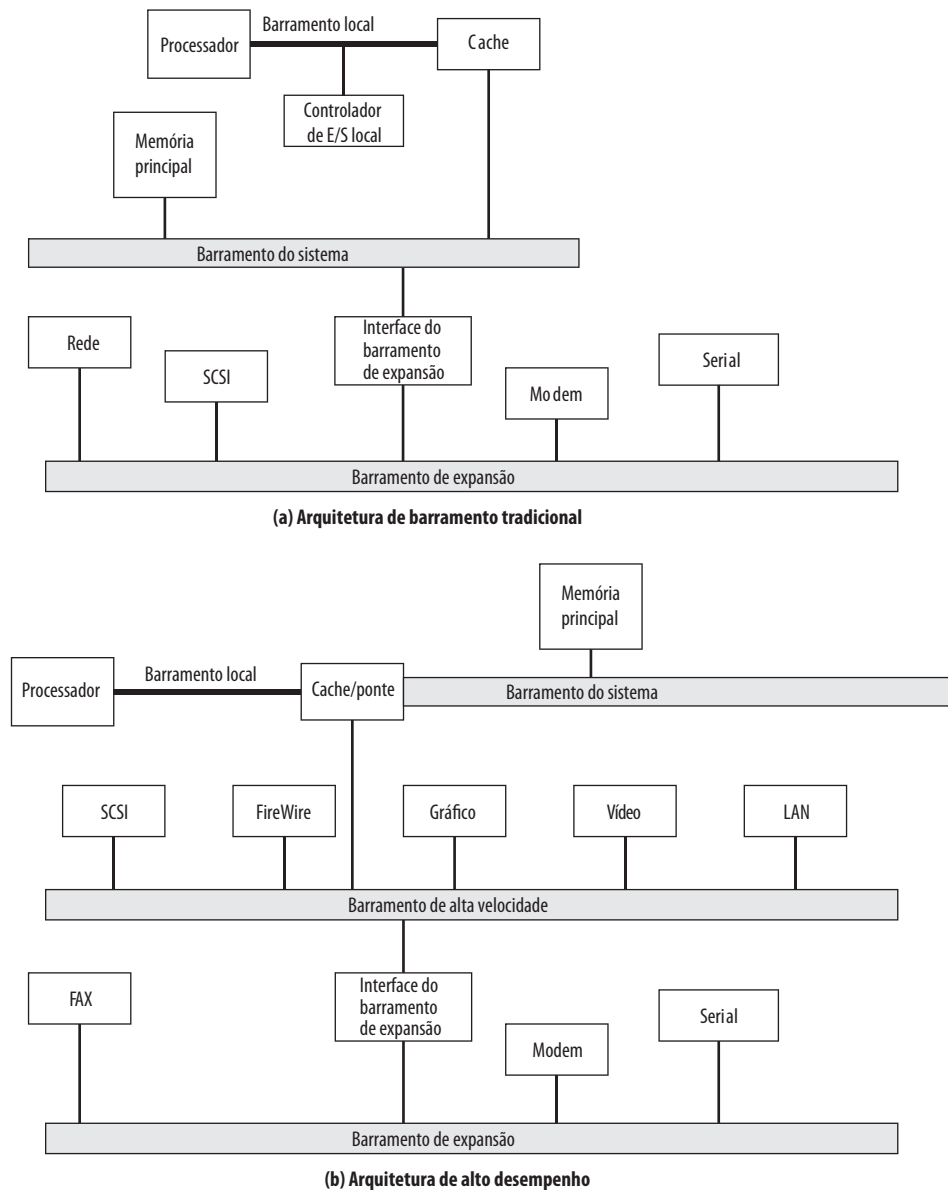
A Figura 3.18a mostra alguns exemplos típicos de dispositivos de E/S que poderiam ser conectados ao barramento de expansão. As conexões de rede incluem redes locais (LANs, do inglês *local area networks*) como uma

Ethernet de 10 Mbps e conexões a redes remotas (WANs, do inglês *wide area networks*), como rede de comutação de pacotes. SCSI (do inglês *small computer system interface*) é um tipo de barramento utilizado para dar suporte a unidades de discos locais e outros periféricos. Uma porta serial poderia ser usada para dar suporte a uma impressora ou um escâner.

Essa arquitetura de barramento tradicional é razoavelmente eficaz, mas começa a fracassar quando um desempenho cada vez maior é visto nos dispositivos de E/S. Em resposta a essas demandas crescentes, uma técnica comum usada pela indústria é montar um barramento de alta velocidade que esteja rigorosamente integrado ao restante do sistema, exigindo apenas uma ponte entre o barramento do processador e o barramento de alta velocidade. Esse arranjo às vezes é conhecido como arquitetura mezanino.

A Figura 3.18b mostra uma realização típica dessa técnica. Novamente, existe um barramento local que conecta o processador a um controlador de cache, que por sua vez, é conectado a um barramento do sistema que admite memória principal. O controlador de cache é integrado a uma ponte, ou dispositivo de armazenamento temporário (buffer), que se conecta ao barramento de alta velocidade. Esse barramento tem suporte para conexões com LANs de alta velocidade, como Fast Ethernet a 100 Mbps, controladores de estação de trabalho de vídeo e gráficos, além de

Figura 3.18 Exemplo de configurações de barramento



controladores de interface com barramentos periféricos locais, incluindo SCSI e FireWire. O último é um arranjo de barramento de alta velocidade projetado especificamente para dar suporte a dispositivos de E/S de alta capacidade. Os dispositivos de menor velocidade ainda são aceitos por um barramento de expansão, por meio de uma interface que faz o armazenamento temporário dos dados do tráfego entre o barramento de expansão e o barramento de alta velocidade.

A vantagem desse arranjo é que o barramento de alta velocidade faz os dispositivos de alta demanda serem integrados mais de perto do processador e, ao mesmo tempo, ele é independente do processador. Assim, as diferenças nas velocidades do processador e do barramento de alta velocidade e as definições da linha de sinal são toleradas. As mudanças na arquitetura do processador não afetam o barramento de alta velocidade, e vice-versa.



Elementos do projeto de barramento

Embora exista uma grande variedade de implementações de barramento diferentes, existem poucos parâmetros ou elementos de projeto básicos que servem para classificar e diferenciar barramentos. A Tabela 3.2 lista os principais elementos.

TIPOS DE BARRAMENTO As linhas de barramento podem ser separadas em dois tipos genéricos: dedicado e multiplexado. Uma linha de barramento dedicada é atribuída permanentemente a uma função ou a um subconjunto físico de componentes de computador.

Um exemplo de dedicação funcional é o uso de linhas de endereço e dados dedicadas separadas, o que é comum em muitos barramentos. Porém, isso não é essencial. Por exemplo, informações de endereço e dados podem ser transmitidas pelo mesmo conjunto de linhas usando uma linha de controle Address Valid. No início de uma transferência de dados, o endereço é colocado no barramento e a linha Address Valid é ativada. Nesse ponto, cada módulo tem um período especificado para copiar o endereço e determinar se ele é o módulo endereçado. O endereço é então removido do barramento, e as mesmas conexões do barramento são usadas para a subsequente transferência de dados de leitura ou escrita. Esse método de uso das mesmas linhas para múltiplas finalidades é conhecido como *multiplexação de tempo*.

A vantagem desse método é o uso de menos linhas, o que economiza espaço e, normalmente, custo. A desvantagem é que um circuito mais complexo é necessário dentro de cada módulo. Além disso, existe uma redução em potencial no desempenho, pois certos eventos que compartilham as mesmas linhas não podem ocorrer em paralelo.

A *dedicação física* refere-se ao uso de múltiplos barramentos, cada um conectando apenas um subconjunto de módulos. Um exemplo típico é o uso de um barramento de E/S para interconectar todos os módulos de E/S; esse barramento é então conectado ao barramento principal por meio de algum tipo de módulo adaptador de E/S. A vantagem em potencial da dedicação física é a alta vazão, pois existe menos disputa pelo barramento. Uma desvantagem é o aumento do tamanho e do custo do sistema.

MÉTODO DE ARBITRAÇÃO Em todos os sistemas, menos os mais simples, mais de um módulo pode precisar do controle do barramento. Por exemplo, um módulo de E/S pode precisar ler ou escrever diretamente na me-

Tabela 3.2 Elementos do projeto de barramento

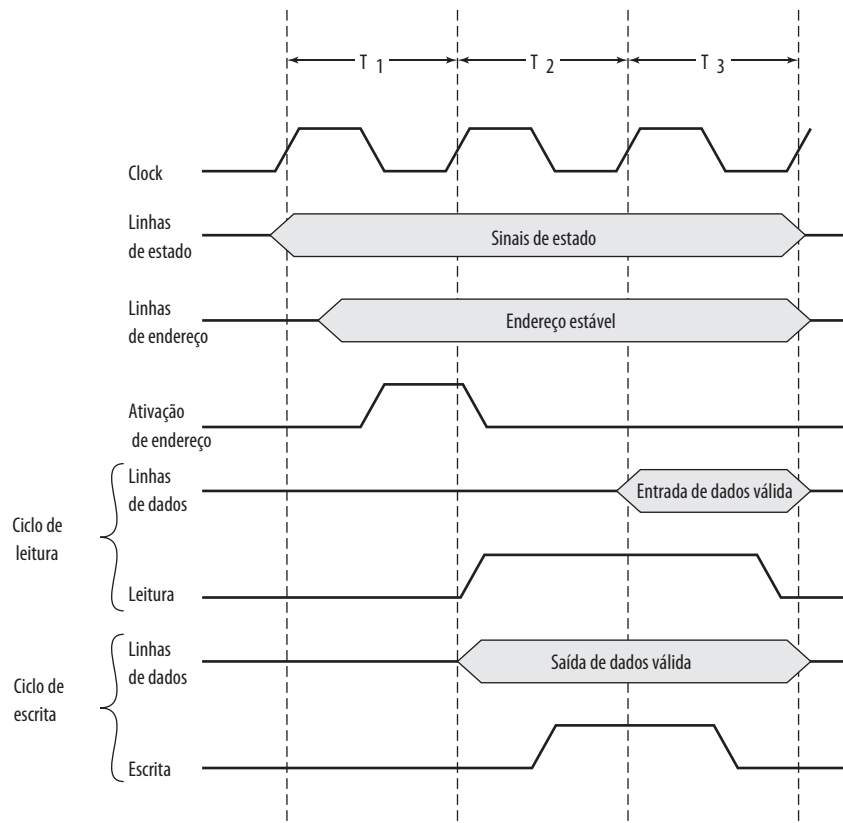
Tipo	Largura do barramento
Dedicado	Endereço
Multiplexado	Dados
Método de arbitração	Tipo de transferência de dados
Centralizado	Leitura
Distribuído	Escrita
Sincronização	Ler-modificar-escrever
Síncrona	Leitura-após-escrita
Assíncrona	Bloco

mória, sem enviar os dados ao processador. Como somente um circuito de cada vez pode fazer transmissões com sucesso pelo barramento, algum método de arbitração é necessário. Os diversos métodos podem ser classificados como centralizados ou distribuídos. Em um esquema centralizado, um único dispositivo de hardware, chamado de *controlador* ou *árbitro de barramento*, é responsável por alocar tempo no barramento. O dispositivo pode ser um módulo separado ou parte do processador. Em um esquema distribuído, não existe um controlador central. Ao invés disso, cada módulo contém lógica de controle de acesso e os módulos atuam juntos para compartilhar o barramento. Com os dois métodos de arbitração, a finalidade é designar um dispositivo, seja o processador ou um módulo de E/S, como mestre. O mestre pode, então, iniciar uma transferência de dados (por exemplo, leitura ou escrita) com algum outro dispositivo, que atua como escravo para essa troca em particular.

TEMPORIZAÇÃO A temporização do barramento refere-se ao modo como os eventos são coordenados no barramento. Os barramentos utilizam temporização síncrona ou assíncrona.

Com a **temporização síncrona**, a ocorrência de eventos no barramento é determinada por um clock. O barramento inclui uma linha de clock, sobre a qual um clock é transmitido como uma sequência regular de 1s e 0s alternados, com a mesma duração. Uma única transmissão 1-0 é conhecida como *ciclo de clock* ou *ciclo de barramento*, e define um *slot* de tempo. Todos os outros dispositivos no barramento podem ler a linha de clock e todos os eventos começam no início de um ciclo de clock. A Figura 3.19 mostra um diagrama de sincronização típico, porém simplificado, para operações síncronas de leitura e escrita (veja no Apêndice 3A uma descrição dos diagramas de sincronização). Outros sinais do barramento podem mudar na transição inicial do sinal de clock (com um pequeno atraso de reação). A maioria dos eventos dura um único ciclo de clock. Nesse exemplo simples, o processador coloca um endereço de memória nas linhas de endereço durante o primeiro ciclo de clock e pode ativar diversas linhas de estado. Quando as linhas de estado tiverem se estabilizado, o processador emite um sinal de ativação de endereço. Para uma operação de leitura, o processador emite um comando de leitura no

Figura 3.19 Temporização de operações de barramento síncronas



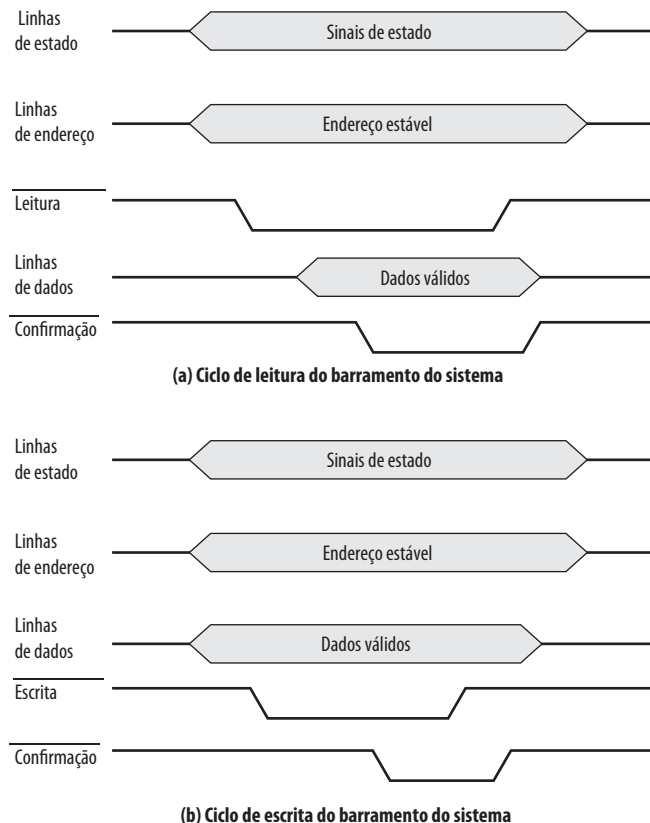
início do segundo ciclo. Um módulo de memória reconhece o endereço e, após um atraso de um ciclo, coloca os dados nas linhas de dados. O processador lê os dados das linhas e retira o sinal de leitura. Para uma operação de escrita, o processador coloca os dados nas linhas no início do segundo ciclo e emite um comando de escrita após as linhas de dados terem se estabilizado. O módulo de memória copia a informação das linhas de dados durante o terceiro ciclo de clock.

Com a **temporização assíncrona**, a ocorrência de um evento em um barramento segue e depende da ocorrência de um evento anterior. No exemplo de leitura simples da Figura 3.20a, o processador coloca sinais de endereço e estado no barramento. Depois de uma pausa para esses sinais se estabilizarem, ele emite um comando de leitura, indicando a presença de sinais válidos de endereço e controle. A memória apropriada decodifica o endereço e responde colocando os dados na linha de dados. Quando as linhas de dados tiverem se estabilizado, o módulo de memória ativa a linha de confirmação para sinalizar ao processador de que os dados estão disponíveis. Quando o mestre tiver lido os dados das linhas de dados, ele desativa o sinal de leitura. Isso faz com que o módulo de memória retire os dados e a linha de confirmação. Finalmente, quando a linha de confirmação for retirada, o mestre remove a informação do endereço.

A Figura 3.20b mostra uma operação de escrita assíncrona simples. Nesse caso, o mestre coloca os dados na linha de dados ao mesmo tempo em que coloca sinais nas linhas de estado e endereço. O módulo de memória responde ao comando de escrita copiando os dados das linhas de dados e, depois, ativando a linha de confirmação. O mestre, então, remove o sinal de escrita e o módulo de memória remove o sinal de confirmação.

A temporização síncrona é mais simples de implementar e testar. Porém, ela é menos flexível que a temporização assíncrona. Como todos os dispositivos em um barramento síncrono estão presos a uma taxa de

Figura 3.20 Temporização de operações de barramento assíncronas

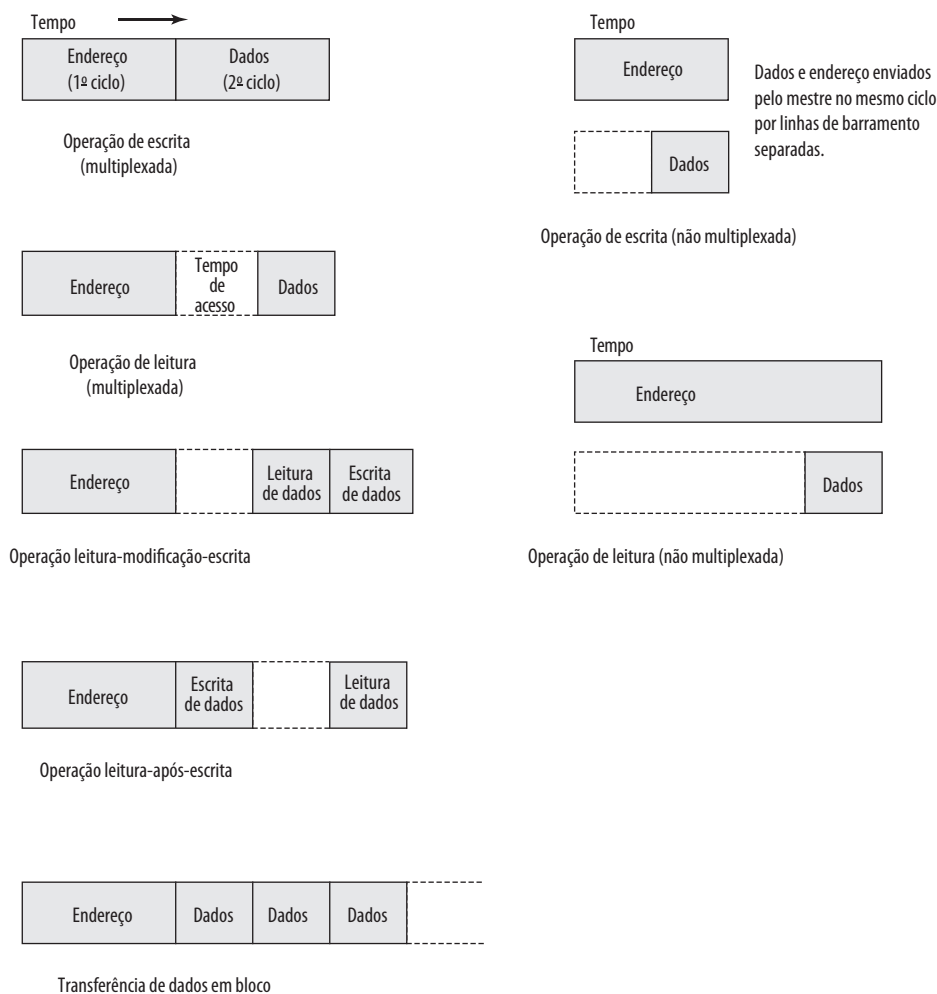


clock fixa, o sistema não pode tirar proveito dos avanços no desempenho do dispositivo. Com a temporização assíncrona, uma mistura de dispositivos lentos e rápidos, usando tecnologia mais antiga e mais nova, pode compartilhar um barramento.

LARGURA DO BARRAMENTO Já explicamos o conceito de largura do barramento. A largura do barramento de dados tem um impacto sobre o desempenho do sistema: quanto mais largo o barramento de dados, maior o número de bits transferidos de cada vez. A largura do barramento de endereço tem um impacto direto sobre a capacidade do sistema: quanto mais largo o barramento de endereço, maior o intervalo de locais que podem ser referenciados.

TIPO DE TRANSFERÊNCIA DE DADOS Finalmente, um barramento permite diversos tipos de transferência de dados, conforme ilustramos na Figura 3.21. Todos os barramentos permitem transferências de escrita (mestre para escravo) e leitura (escravo para mestre). No caso de um barramento de endereço/dados multiplexado, o barramento primeiro é usado para especificar o endereço e depois para transferir os dados. Para uma operação de leitura, normalmente existe uma espera enquanto os dados estão sendo buscados do escravo para serem colocados no barramento. Para uma leitura ou uma escrita, também pode haver uma espera se for necessário passar pela arbitragem para obter controle do barramento para o restante da operação (ou seja, apoderar-se do barramento para solicitar uma leitura ou escrita, depois apoderar-se do barramento novamente para realizar uma leitura ou escrita).

Figura 3.21 Tipos de transferência de dados do barramento



No caso de barramentos de endereço e dados dedicados, o endereço é colocado no barramento de endereço e permanece lá enquanto os dados são colocados no barramento de dados. Para uma operação de escrita, o mestre coloca os dados no barramento de dados assim que o endereço tiver sido estabilizado e o escravo tiver a oportunidade de reconhecer seu endereço. Para uma operação de leitura, o escravo coloca os dados no barramento assim que tiver reconhecido seu endereço e apanhado os dados.

Alguns barramentos permitem diversas outras operações de combinação. Uma operação leitura-modificação-escrita é simplesmente uma leitura seguida imediatamente por uma escrita no mesmo endereço. O endereço só é transmitido uma vez, no início da operação. A operação inteira normalmente é indivisível, para impedir qualquer acesso ao elemento de dados por outros mestres de barramento (*bus masters*) em potencial. A finalidade principal dessa capacidade é proteger os recursos de memória compartilhada em um sistema de multiprogramação (veja Capítulo 8).

Leitura-após-escrita é uma operação indivisível, que consiste em uma escrita seguida imediatamente por uma leitura do mesmo endereço. A operação de leitura pode ser realizada para fins de verificação.

Alguns sistemas de barramento também permitem uma transferência de dados em bloco. Nesse caso, um ciclo de endereço é seguido por n ciclos de dados. O primeiro item de dados é transferido de ou para o endereço especificado; os itens de dados restantes são transferidos de ou para endereços subsequentes.



3.5 PCI

O barramento PCI (PCI do inglês *peripheral component interconnect*) é um barramento de grande largura de banda, independente de processador, que pode funcionar como um mezanino ou barramento periférico. Em comparação com outras especificações de barramento comuns, o PCI oferece melhor desempenho de sistema para subsistemas de E/S de alta velocidade (por exemplo, adaptadores de vídeo gráficos, controladores de interface de rede, controladores de disco e assim por diante). O padrão atual permite o uso de até 64 linhas de dados a 66 MHz, para uma taxa de transferência bruta de 528 MB/s, ou 4,224 Gbps. Mas não é apenas a velocidade alta que torna o PCI atraente. O PCI é projetado especificamente para atender economicamente os requisitos de E/S dos sistemas modernos; ele requer muito poucos chips para ser implementado e admite outros barramentos conectados a ele.

A Intel começou a trabalhar no PCI em 1990, para seus sistemas baseados no Pentium, e logo lançou todas as patentes ao domínio público, promovendo a criação de uma associação da indústria, o PCI *Special Interest Group* (PCI SIG), para desenvolver melhor e manter a compatibilidade das especificações do PCI. O resultado é que o PCI tem sido bastante adotado e está encontrando uso cada vez maior nos sistemas de computadores pessoais, estações de trabalho e servidores. Como a especificação é de domínio público e é adotada por uma grande parte da indústria de microprocessadores e periféricos, produtos PCI criados por diferentes fornecedores são compatíveis.

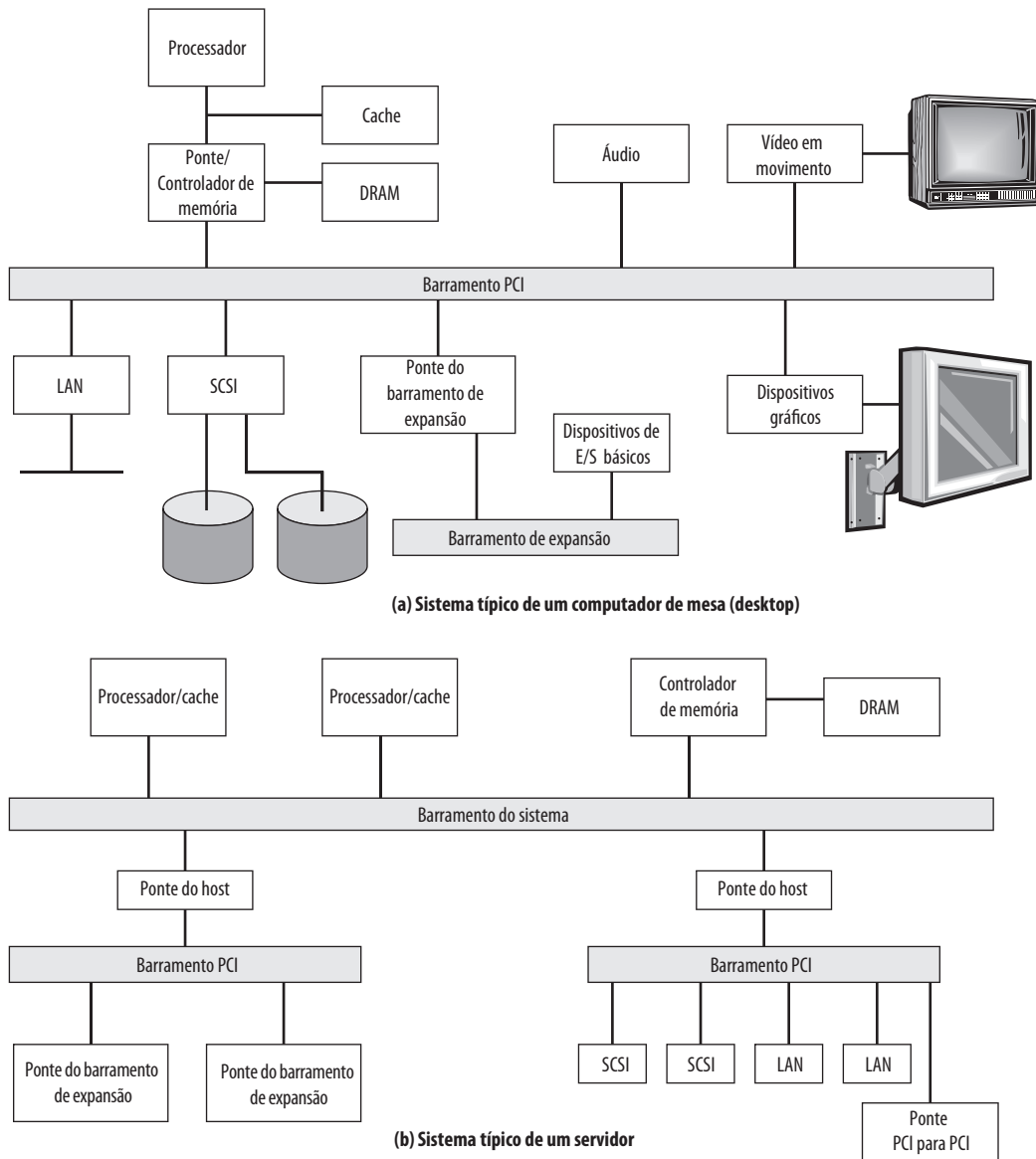
O PCI foi projetado para admitir uma série de configurações baseadas em microprocessadores, incluindo sistemas de processador único e múltiplo. Consequentemente, ela oferece um conjunto de funções de uso geral e utiliza a temporização síncrona e um esquema de arbitragem centralizada.

A Figura 3.22a mostra um uso típico do barramento PCI em um sistema de único processador. Um controlador de DRAM e ponte combinados oferecem um acoplamento adequado com o processador, possibilitando a transferência de dados em altas velocidades. A ponte atua como um buffer de dados, de modo que a velocidade do barramento PCI pode diferir da capacidade de transferência do E/S do processador. Em um sistema multiprocessador (Figura 3.22b), uma ou mais configurações PCI podem ser conectadas por pontes ao barramento do sistema do processador. O barramento do sistema admite apenas as unidades de processador/cache, memória principal e pontes PCI. Novamente, o uso de pontes mantém o PCI independente da velocidade do processador, enquanto oferece a capacidade de receber e enviar dados rapidamente.



Estrutura de barramento

O barramento PCI pode ser configurada como um barramento de 32 ou 64 bits. A Tabela 3.3 define as 49 linhas de sinal obrigatórias para o barramento PCI. Estas são divididas nos seguintes grupos funcionais:

Figura 3.22 Exemplo de configurações PCI

- **Pinos do sistema:** incluem os pinos de clock e reset.
- **Pinos de endereços e de dados:** incluem 32 linhas que são multiplexadas no tempo para endereços e dados. As outras linhas nesse grupo são usadas para interpretar e validar as linhas de sinal que carregam os endereços e dados.
- **Pinos de controle da interface:** controlam a temporização de transações e oferecem coordenação entre iniciadores e destinos.
- **Pinos de arbitração:** diferente das outras linhas de sinal PCI, estas não são linhas compartilhadas. Em vez disso, cada mestre PCI tem seu próprio par de linhas de arbitração que a conectam diretamente ao arbitrador do barramento PCI.
- **Pinos de erros:** usado para indicar erros de paridade e outros.

Tabela 3.3 Linhas de sinal do PCI obrigatórias

Designação	Tipo	Descrição
Pinos do sistema		
CLK	in	Fornece a temporização para todas as transações e todas as entradas são amostradas na transição de subida. São admitidas taxas de clock de até 33 MHz.
RST#	in	Inicializa todos os registradores específicos do PCI, sequenciadores e sinais.
Pinos de endereço e de dados		
AD[31::0]	t/s	Linhas multiplexadas usadas para endereços e dados.
C/BE[3::0]#	t/s	Sinais multiplexados para comandos de barramento e para habilitação de bytes de dados. Durante a fase de dados, as linhas indicam qual das quatro pistas transporta dados significativos.
PAR	t/s	Fornece paridade par pelas linhas AD e C/BE no ciclo de clock seguinte. O mestre envia o sinal PAR para as fases de escrita de dados e de endereço; o destino envia o sinal PAR para as fases de leitura de dados.
Pinos de controle de interface		
FRAME#	s/t/s	Controlado pelo mestre corrente para indicar o início e a duração de uma transação. Ele é ativado no início e desativado quando o iniciador estiver pronto para começar a última fase de dados.
IRDY#	s/t/s	Iniciador pronto, controlado pelo mestre de barramento corrente (iniciador da transação). Durante uma leitura, indica que o mestre está preparado para receber dados; durante uma escrita, indica que os dados válidos estão presentes em AD.
TRDY#	s/t/s	Destino pronto, controlado pelo destino (dispositivo selecionado). Durante uma leitura, indica que os dados válidos estão presentes em AD; durante uma escrita, indica que o destino está pronto para receber dados.
STOP#	s/t/s	Indica que o destino corrente deseja que o iniciador interrompa a transação atual.
IDSEL	in	Seleção de dispositivo de inicialização. Usado como uma seleção de chip durante as transações de leitura e escrita de configuração.
DEVSEL#	in	Seleção de dispositivo. Ativado pelo destino quando ele tiver reconhecido seu endereço. Indica ao iniciador corrente se algum dispositivo foi selecionado.
Pinos de arbitração		
REQ#	t/s	Indica ao arbitrador que esse dispositivo requer o uso do barramento. Essa é uma linha ponto a ponto específica do dispositivo.
GNT#	t/s	Indica ao dispositivo que o arbitrador concedeu acesso ao barramento. Essa é uma linha ponto a ponto específica do dispositivo.
Pinos de erro		
PERR#	s/t/s	Erro de paridade. Indica que um erro de paridade de dados é detectado por um destino durante uma fase de escrita de dados ou por um iniciador durante uma fase de leitura de dados.
SERR#	o/d	Erro do sistema. Pode ser enviado por qualquer dispositivo para relatar erros de paridade de endereço e erros críticos diferentes de paridade.

Além disso, a especificação PCI define 51 linhas de sinal opcionais (Tabela 3.4), divididas nos seguintes grupos funcionais:

- **Pinos de interrupção:** estes são disponíveis para dispositivos PCI que precisam gerar solicitações de serviço (interrupções). Assim como os pinos de arbitração, estes não são linhas compartilhadas. Em vez disso, cada dispositivo PCI tem sua própria linha ou linhas de interrupção para um controlador de interrupção.
- **Pinos de suporte à cache:** esses pinos são necessários para dar suporte a uma memória no PCI que possa ser armazenada em uma memória cache do processador ou de outro dispositivo. Esses pinos permitem os protocolos de cache *snoopy* (veja no Capítulo 18 uma discussão sobre esses protocolos).
- **Pinos de extensão de barramento de 64 bits:** incluem 32 linhas que são multiplexadas no tempo para endereços e dados, e que são combinadas com linhas de endereço/dados obrigatórias para formar um barramento de endereço/dados de 64 bits. Outras linhas nesse grupo são usadas para interpretar e validar

Tabela 3.4 Linhas de sinal PCI opcionais

Designação	Tipo	Descrição
Pinos de interrupção		
INTA#	o/d	Usado para requisitar uma interrupção.
INTB#	o/d	Usado para requisitar uma interrupção; só tem significado para dispositivos multifuncionais.
INTC#	o/d	Usado para requisitar uma interrupção; só tem significado para dispositivos multifuncionais.
INTD#	o/d	Usado para requisitar uma interrupção; só tem significado para dispositivos multifuncionais.
Pinos de suporte de cache		
SBO#	in/out	Recuo de <i>snoop</i> . Indica um acerto, em uma linha modificada na cache.
SDONE	in/out	<i>Snoop</i> completo. Indica o estado do <i>snoop</i> para o endereço corrente. Ativado quando o <i>snoop</i> tiver sido concluído.
Pinos de extensão de barramento de 64 bits		
AD[63::32]	t/s	Linhas multiplexadas para endereço e dados para estender o barramento até 64 bits.
C/BE[7::4]#	t/s	Sinais multiplexados para comandos de barramento e para habilitação de bytes de dados. Durante a fase de endereço, as linhas fornecem comandos de barramento adicionais. Durante a fase de dados, as linhas indicam qual das quatro vias de bytes da extensão carregam dados significativos.
REQ64#	s/t/s	Usado para requisitar transferência de 64 bits.
ACK64#	s/t/s	Indica que o destino está querendo realizar uma transferência de 64 bits.
PAR64	t/s	Oferece paridade par pelas linhas AD e C/BE estendidas um ciclo de clock adiante.
JTAG/pinos de testes		
TCK	in	Clock de teste. Usado para informação de estado do clock e teste de dados entrando e saindo do dispositivo durante varredura de fronteira.
TDI	in	Entrada de teste. Usado para deslocar dados e instruções de teste em forma serial para dentro do dispositivo.
TDO	out	Saída de teste. Usado para deslocar dados e instruções de teste em forma serial para fora do dispositivo.
TMS	in	Seleção de modo de teste. Usado para controlar o estado do controlador da porta de acesso de teste.
TRST#	in	Reset de teste. Usado para inicializar o controlador da porta de acesso de teste.

in Sinal apenas de entrada**out** Sinal apenas de saída**t/s** Bidirecional, tri-state. Sinal de E/S**s/t/s** Sinal tri-state sustentado, controlado apenas por um proprietário de cada vez**o/d** Dreno aberto: permite que múltiplos dispositivos compartilhem como um wire-OR**#** O estado ativo do sinal ocorre na voltagem baixa

as linhas de sinal que transportam endereços e dados. Finalmente, existem duas linhas que ativam dois dispositivos PCI para combinar como o uso da capacidade de 64 bits.

- **JTAG/pinos de testes:** essas linhas de sinal admitem procedimentos de teste definidos no padrão IEEE 1149.1.



Comandos PCI

A atividade do barramento ocorre na forma de transações entre um iniciador, ou mestre, e um destino. Quando um mestre do barramento adquire o controle do barramento, ele determina o tipo da transação que ocorrerá em seguida. Durante a fase de endereço da transação, as linhas C/BE são usadas para sinalizar o tipo de transação. Os comandos são os seguintes:

- *Interrupt Acknowledge* (confirmação de interrupção).
- *Special Cycle* (ciclo especial).
- *I/O Read* (leitura de E/S).
- *I/O Write* (escrita de E/S).

- *Memory Read* (leitura de memória).
- *Memory Read Line* (linha de leitura de memória).
- *Memory Read Multiple* (leitura de memória múltipla).
- *Memory Write* (escrita de memória).
- *Memory Write and Invalidate* (escrita de memória e invalidação).
- *Configuration Read* (leitura de configuração).
- *Configuration Write* (escrita de configuração).
- *Dual Address Cycle* (ciclo de endereço duplo).

Interrupt Acknowledge é um comando de leitura voltado para o dispositivo, que funciona como um controlador de interrupção no barramento PCI. As linhas de endereço não são usadas durante a fase de endereço e as linhas *byte enable* (habilitação de byte) indicam o tamanho do identificador de interrupção a ser retornado.

O comando *Special Cycle* é usado pelo iniciador para transmitir uma mensagem a um ou mais destinos.

Os comandos *I/O Read* e *Write* são usados para transferir dados entre o iniciador e um controlador de E/S. Cada dispositivo de E/S tem seu próprio espaço de endereços, e as linhas de endereço são usadas para indicar um dispositivo em particular e para especificar os dados a serem transferidos de ou para esse dispositivo. O conceito de endereços de E/S é explorado no Capítulo 7.

Os comandos de leitura e escrita de memória são usados para especificar a transferência de uma porção de dados, ocupando um ou mais ciclos de clock. A interpretação desses comandos depende do controlador de memória no barramento: se o PCI suporta ou não o protocolo PCI para transferências entre memória e cache. Se suporta a transferência de dados de e para a memória, esta é normalmente feita com linhas de cache, ou blocos.³ Os três comandos de leitura de memória têm os usos esboçados na Tabela 3.5. O comando *Memory Write* é usado para transferir dados em um ou mais ciclos de dados para a memória. O comando *Memory Write and Invalidate* transfere dados em um ou mais ciclos para a memória. Além disso, ele garante que pelo menos uma linha de cache será escrita. Esse comando admite a função da cache de escrever de volta uma linha para a memória.

Os dois comandos de configuração permitem que um mestre leia e atualize parâmetros de configuração em um dispositivo conectado ao PCI. Cada dispositivo PCI pode incluir até 256 registradores internos, que são usados durante a inicialização do sistema para configurar esse dispositivo.

O comando *Dual Address Cycle* é usado por um iniciador para indicar que está usando o endereçamento com 64 bits.



Transferências de dados

Cada transferência de dados no barramento PCI é uma única transação que consiste em uma fase de endereço e uma ou mais fases de dados. Nesta discussão, ilustramos uma operação típica de leitura; uma operação de escrita prossegue de modo semelhante.

Tabela 3.5 Interpretação dos comandos de leitura PCI

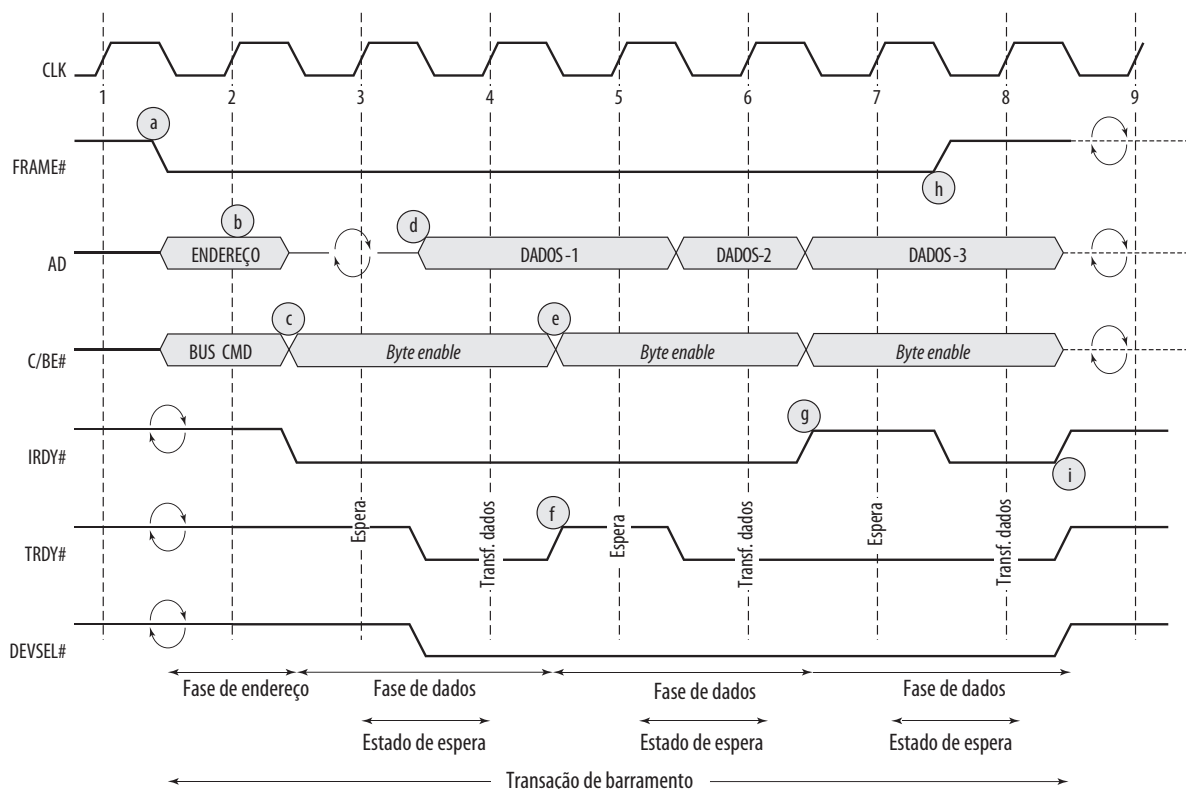
Tipo de comando de leitura	Para memória cacheável	Para memória não cacheável
<i>Memory Read</i>	Transferência de metade ou menos de uma linha de cache	Transferência de dados durante 2 ciclos de clock ou menos
<i>Memory Read Line</i>	Transferência de mais de metade de uma linha de cache para três linhas de cache	Transferência de dados durante 3 a 12 ciclos de clock
<i>Memory Read Multiple</i>	Transferência de mais de três linhas de cache	Transferência de dados por mais de 12 ciclos de clock

³ Os princípios fundamentais da memória cache são descritos no Capítulo 4; os protocolos de cache baseados em barramento são descritos no Capítulo 17.

A Figura 3.23 mostra a temporização da transação de leitura. Todos os eventos são sincronizados para as transições de descida do clock, que ocorrem no meio do ciclo de clock. Os dispositivos do barramento verificam as linhas do barramento na transição de subida, no início de um ciclo do barramento. Os eventos rotulados no diagrama são descritos a seguir:

- Quando um mestre de barramento tiver obtido o controle do barramento, ele pode iniciar a transação ativando FRAME. Essa linha permanece ativada até que o iniciador esteja pronto para completar a última fase de dados. O iniciador também coloca o endereço inicial no barramento de endereço e o comando de leitura nas linhas C/BE.
- No início do segundo ciclo de clock, o dispositivo de destino reconhecerá seu endereço nas linhas AD.
- O iniciador para de enviar informações pelo barramento AD. Um ciclo de *turnaround* (indicado por duas setas circulares) é necessário em todas as linhas de sinal que podem ser controladas por mais de um dispositivo, de modo que a remoção do sinal de endereço preparará o barramento para ser usado pelo dispositivo de destino. O iniciador muda a informação nas linhas C/BE para designar quais linhas AD devem ser usadas para transferência para os dados atualmente endereçados (de 1 a 4 bytes). O iniciador também ativa IRDY para indicar que está pronto para o primeiro item de dado.
- O destino selecionado ativa DEVSEL para indicar que reconheceu seu endereço e responderá. Ele coloca os dados solicitados nas linhas AD e ativa TRDY para indicar que dados válidos estão presentes no barramento.
- O iniciador lê os dados no início do quarto ciclo de clock e muda as linhas *byte enable* conforme a necessidade, em preparação para a próxima leitura.
- Nesse exemplo, o destino precisa de algum tempo para se preparar para o segundo bloco de dados para transmissão. Portanto, ele desativa TRDY para sinalizar ao iniciador que não haverá novos dados durante o ciclo que chega. Com isso, o iniciador não lê as linhas de dados no início do quinto ciclo de clock e não muda o *byte enable* durante esse ciclo. O bloco de dados é lido no início do sexto ciclo de clock.

Figura 3.23 Operação de leitura no barramento PCI



- g. Durante o sexto ciclo de clock, o destino coloca o terceiro item de dados no barramento. Porém, neste exemplo, o iniciador ainda não está pronto para ler o item de dados (por exemplo, ele tem uma condição de buffer temporário cheio). Portanto, ele desativa o IRDY fazendo o destino manter o terceiro item de dados no barramento por mais um ciclo de clock.
- h. O iniciador sabe que a terceira transferência de dados é a última e, portanto, desativa FRAME para sinalizar ao destino que essa é a última transferência de dados. Ele também ativa IRDY para sinalizar que está pronto para completar essa transferência.
- i. O iniciador desativa IRDY, retornando o barramento ao estado ocioso, e o destino desativa TRDY e DEVSEL.



Arbitração

O barramento PCI utiliza um esquema de arbitração centralizado, síncrono, em que cada mestre tem um único sinal de requisição (REQ) e concessão, ou *grant* (GNT). Essas linhas de sinal são conectadas a um árbitro central (Figura 3.24) e um protocolo simples de requisição-concessão é utilizado para conceder acesso ao barramento.

A especificação PCI não dita um algoritmo de arbitração em particular. O árbitro pode usar uma técnica de “atender a primeira requisição que chegar”, uma técnica *round-robin* ou algum tipo de esquema de prioridade. Um mestre PCI precisa arbitrar para cada transação que deseja realizar, onde uma única transação consiste em uma fase de endereço seguida por uma ou mais fases de dados contíguas.

A Figura 3.25 é um exemplo no qual o controle de barramento é arbitrado entre dois dispositivos A e B. A seguinte sequência ocorre:

- a. Em algum ponto antes do início do primeiro ciclo de clock, A terá ativado seu sinal REQ. O árbitro verifica esse sinal no início do primeiro ciclo de clock.
- b. Durante o primeiro ciclo de clock, B solicita o uso do barramento ativando seu sinal REQ.
- c. Ao mesmo tempo, o árbitro ativa GNT-A para conceder acesso ao barramento para A.
- d. O mestre de barramento A verifica GNT-A no início do segundo período de clock e descobre que ele recebeu acesso ao barramento. Ele também verifica IRDY e TRDY desativados, indicando que o barramento está ocioso. Com isso, ele ativa FRAME e coloca a informação do endereço no barramento de endereço e o comando no barramento C/BE (não mostrado). Ele também mantém o sinal REQ-A, pois tem uma segunda transação para realizar em seguida.
- e. O árbitro do barramento verifica todas as linhas REQ no início do terceiro ciclo de clock e toma uma decisão de arbitração para conceder o barramento a B para a próxima transação. Depois, ele ativa GNT-B e desativa GNT-A. B não poderá usar o barramento até que ele retorne a um estado ocioso.
- f. A desativa FRAME para indicar que a última (e única) transferência de dados está em andamento. Ele coloca os dados no barramento de dados e sinaliza o destino com IRDY. O destino lê os dados no início do próximo ciclo de clock.
- g. No início do quinto ciclo de clock, B verifica IRDY e FRAME desativados e, portanto, consegue tomar o controle do barramento, ativando FRAME. Ele também desativa sua linha REQ, pois só deseja realizar uma transação.

Figura 3.24 Árbitro de barramento PCI

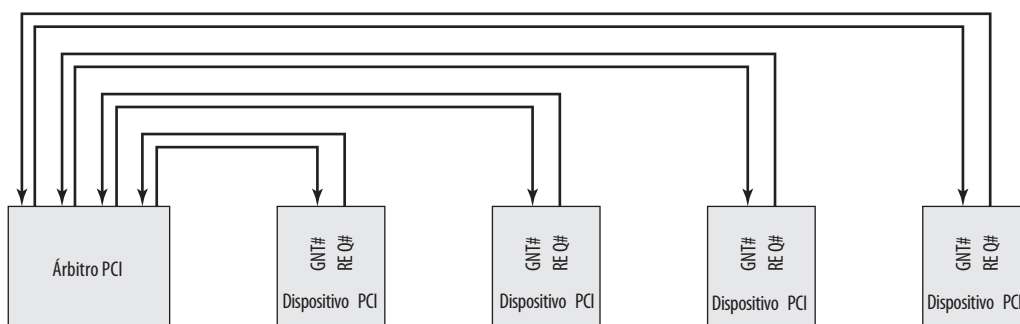
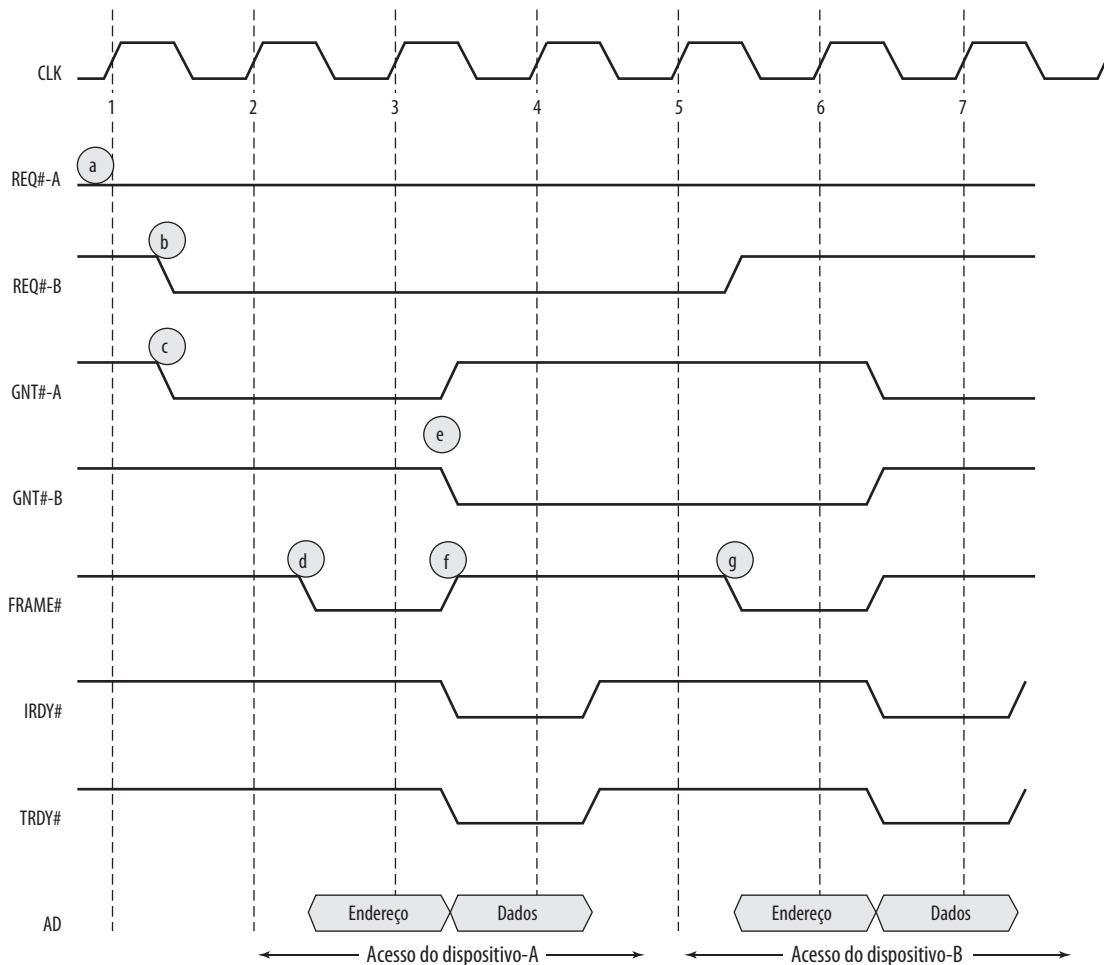


Figura 3.25 Arbitração de barramento PCI entre dois mestres

Em seguida, o mestre A recebe acesso ao barramento para sua próxima transação.

Observe que a arbitração pode ocorrer ao mesmo tempo em que o mestre de barramento atual está realizando uma transferência de dados. Portanto, nenhum ciclo do barramento é perdido ao se realizar a arbitração. Isso é conhecido como *arbitração oculta*.

3.6 Leitura recomendada e sites Web

A descrição mais clara sobre barramento PCI é encontrada em um livro de Shanley e Anderson (1999)^b. Em Abbot (2004)^c também existe muita informação sólida sobre barramento PCI.

Sites Web recomendados

PCI Special Interest Group: informações sobre especificações e produtos PCI.

PCI Pointers: links para vendedores de PCI e outras fontes de informação.

Principais termos, perguntas de revisão e problemas

Principais termos

barramento de endereço	arbitração distribuída	registrador de endereço de memória (MAR)
temporização assíncrona	ciclo de instrução	
barramento	execução de instrução	registrador de buffer de memória (MBR)
arbitração de barramento	busca de instrução	componente periférico
largura de barramento	interrupção	interconexão (PCI)
arbitração centralizada	tratador de interrupção	temporização síncrona
barramento de dados		
interrupção desabilitada	rotina de tratamento de interrupção	barramento do sistema

Perguntas de revisão

- 3.1 Que categorias gerais de funções são especificadas pelas instruções do computador?
- 3.2 Liste e defina resumidamente os estados possíveis que definem a execução de uma instrução.
- 3.3 Liste e defina resumidamente duas técnicas para lidar com múltiplas interrupções.
- 3.4 Que tipos de transferências a estrutura de interconexão de um computador (por exemplo, barramento) precisa aceitar?
- 3.5 Qual é o benefício de usar a arquitetura de barramento múltiplo em comparação com uma arquitetura de barramento único?
- 3.6 Liste e defina resumidamente os grupos funcionais das linhas de sinal para o barramento PCI.

Problemas

- 3.1 A máquina hipotética da Figura 3.4 também tem duas instruções de E/S:

0011 = Carregar AC de E/S

0011 = Armazenar AC em E/S

Nesses casos, o endereço de 12 bits identifica um dispositivo de E/S em particular. Mostre a execução do programa (usando o formato da Figura 3.5) para o programa a seguir:

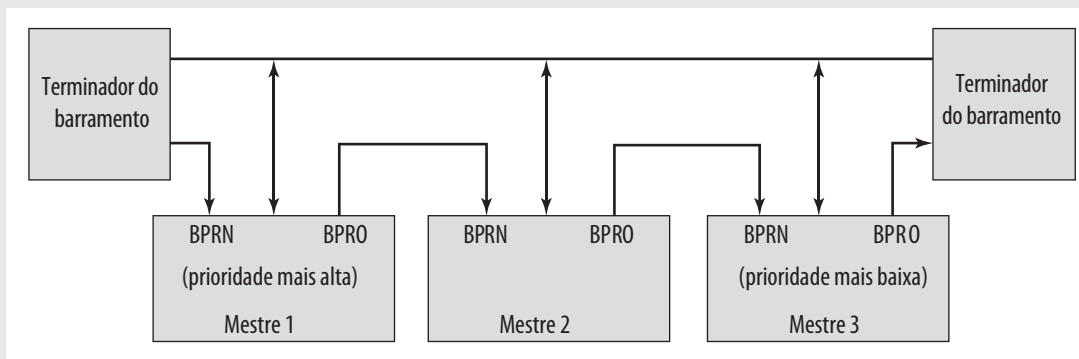
1. Carregar AC do dispositivo 5.
2. Somar o conteúdo do local de memória 940.
3. Armazenar AC no dispositivo 6.

Suponha que o próximo valor apanhado do dispositivo 5 seja 3 e que o local 940 contenha o valor 2.

- 3.2 A execução do programa da Figura 3.5 é descrita no texto usando seis etapas. Expanda essa descrição para mostrar o uso do MAR e do MBR.
- 3.3 Considere um microprocessador de 32 bits hipotético com instruções de 32 bits, compostas de dois campos: o primeiro byte contém o *opcode* e o restante, o operando imediato ou o endereço de um operando.
 - a. Qual é a capacidade de memória máxima endereçável diretamente (em bytes)?
 - b. Discuta o impacto sobre a velocidade do sistema se o barramento do microprocessador tiver
 1. um barramento de endereço local de 32 bits e um barramento de dados local de 16 bits, ou
 2. um barramento de endereço local de 16 bits e um barramento de dados local de 16 bits.
 - c. Quantos bits são necessários para o contador de programa e o registrador de instrução?
- 3.4 Considere um microprocessador hipotético gerando um endereço de 16 bits (por exemplo, suponha que o contador de programa e os registradores de endereço tenham 16 bits de largura) e tendo um barramento de dados de 16 bits.
 - a. Qual é o espaço de endereço de memória máximo que o processador pode acessar diretamente se estiver conectado a uma “memória de 16 bits”?
 - b. Qual é o espaço de endereço de memória máximo que o processador pode acessar diretamente se estiver conectado a uma “memória de 8 bits”?
 - c. Que recursos arquiteturais permitirão que esse microprocessador acesse um “espaço de E/S” separado?
 - d. Se uma instrução de entrada e saída pode especificar um número de porta de E/S de 8 bits, quantas portas de E/S de 8 bits o microprocessador pode aceitar? Quantas portas de E/S de 16 bits? Explique.

- 3.5** Considere um microprocessador de 32 bits, com um barramento de dados de 16 bits, controlado por um clock de entrada de 8 MHz. Suponha que esse microprocessador tenha um ciclo de barramento cuja duração mínima seja igual a 4 ciclos de clock. Qual é a taxa de transferência de dados máxima pelo barramento que esse microprocessador pode sustentar, em bytes/s? Para aumentar seu desempenho, seria melhor tornar seu barramento de dados externo de 32 bits ou dobrar a frequência de clock externa fornecida ao microprocessador? Informe quaisquer outras suposições que você faça e explique. *Dica:* determine o número de bytes que podem ser transferidos por ciclo de barramento.
- 3.6** Considere um sistema de computação que contenha um módulo de E/S controlando um teletipo teclado/impressora simples. Os registradores a seguir estão contidos no processador e conectados diretamente ao barramento do sistema:
- INPR: Registrador de entrada, 8 bits
 - OUTR: Registrador de saída, 8 bits
 - FGI: Flag de entrada, 1 bit
 - FGO: Flag de saída, 1 bit
 - IEN: Ativação de interrupção, 1 bit
- A entrada de teclado do teletipo e a saída da impressora no teletipo são controladas pelo módulo de E/S. O teletipo é capaz de codificar um símbolo alfanumérico para uma palavra de 8 bits e decodificar uma palavra de 8 bits para um símbolo alfanumérico.
- a. Descreva como o processador, usando os quatro primeiros registradores listados neste problema, pode alcançar a E/S com o teletipo.
 - b. Descreva como a função pode ser realizada de forma mais eficiente também empregando a IEN.
- 3.7** Considere dois microprocessadores tendo barramentos de dados externos de 8 e 16 bits, respectivamente. Os dois processadores são idênticos em outros aspectos e seus ciclos de barramento ocupam o mesmo espaço.
- a. Suponha que todas as instruções e operandos tenham 2 bytes de extensão. Por que fator diferem as taxas de transferência de dados máximas?
 - b. Repita considerando que metade dos operandos e instruções tenham 1 byte de extensão.
- 3.8** A Figura 3.26 indica um esquema de arbitração distribuído que pode ser usado com um barramento obsoleto conhecido como Multibus I. Os agentes são encadeados fisicamente em forma de margarida em ordem de prioridade. O agente mais à esquerda no diagrama recebe um sinal *bus priority in* (BPRN) indicando que nenhum agente com prioridade mais alta deseja o barramento. Se o agente não requisitar o barramento, ele ativa sua linha *bus priority out* (BPRO). No início de um ciclo de clock, qualquer agente pode requisitar o controle do barramento reduzindo sua linha BPRO. Isso abaixa a linha BPRN do próximo agente na cadeia, que por sua vez, precisa abaixar sua linha BPRO. Assim, o sinal propaga a extensão da cadeia. Ao final dessa reação em cadeia, deverá haver apenas um agente cujo BPRN está ativado e cujo BPRO não está. Esse agente tem prioridade. Se, no início de um ciclo de barramento, o barramento não estiver ocupado (BUSY inativo), o agente que tem prioridade pode apanhar o controle do barramento ativando a linha BUSY.
- É preciso um certo tempo para que o sinal BPR se propague do agente de prioridade mais alta para o de prioridade mais baixa. Esse tempo deve ser menor que o ciclo de clock? Explique.
- 3.9** O barramento SBI do VAX utiliza um esquema de arbitração distribuído e síncrono. Cada dispositivo SBI (ou seja, processador, memória, módulo de E/S) tem uma prioridade exclusiva e recebe uma linha de requisição de transferência (TR) exclusiva. O SBI tem 16 dessas linhas (TR0, TR1,...,TR15), com TR0 tendo a prioridade mais alta. Quando um dispositivo deseja usar o barramento, ele coloca uma reserva para um *slot* de

Figura 3.26 Arbitração distribuída do barramento Multibus I



tempo futuro ativando sua linha TR durante o *slot* de tempo atual. Ao final do *slot* de tempo atual, cada dispositivo com uma reserva pendente examina as linhas TR; o dispositivo com prioridade mais alta com uma reserva utiliza o próximo *slot* de tempo.

Um máximo de 17 dispositivos pode ser conectados ao barramento. O dispositivo com prioridade 16 não tem linha TR. Por que não?

- 3.10** No SBI do VAX, o dispositivo com menor prioridade normalmente tem o tempo médio de espera mais baixo. Por esse motivo, o processador normalmente recebe a prioridade mais baixa no SBI. Por que o dispositivo de prioridade 16 normalmente tem o tempo médio de espera mais baixo? Sob que circunstâncias isso não seria verdadeiro?
- 3.11** Para uma operação de leitura síncrona (Figura 3.19), o módulo de memória precisa colocar os dados no barramento suficientemente antes da transição de descida do sinal Read para permitir o estabelecimento do sinal. Suponha que o barramento do microprocessador tenha um clock de 10 MHz e que o sinal Read comece a cair no meio da segunda metade de T_3 .
- Determine a extensão do ciclo de instrução de leitura de memória.
 - Quando, no máximo, os dados da memória devem ser colocados no barramento? Permita 20 ns para o estabelecimento das linhas de dados.
- 3.12** Considere um microprocessador que tenha uma sincronização de leitura de memória conforme mostra a Figura 3.19. Após alguma análise, um projetista determina que a memória não consegue oferecer dados de leitura a tempo por cerca de 180 ns.
- Quantos estados de espera (ciclos de clock) precisam ser inseridos para a operação apropriada do sistema se a frequência do sinal de clock do barramento é de 8 MHz?
 - Para forçar os estados de espera, uma linha de estado Ready é empregada. Quando o processador tiver emitido um comando Read, ele precisa esperar até que a linha Ready seja ativada antes de tentar ler dados. Em que intervalo de tempo devemos manter a linha Ready baixa a fim de forçar o processador a inserir o número requisitado de estados de espera?
- 3.13** Um microprocessador tem uma sincronização de escrita na memória conforme mostra a Figura 3.19. Seu fabricante especifica que a largura do sinal Write pode ser determinada por $T - 50$, onde T é o período de clock em ns.
- Que largura devemos esperar para o sinal Write se a frequência do sinal de clock do barramento é de 5 MHz?
 - O manual do microprocessador especifica que os dados permanecem válidos por 20 ns após a transição de descida do sinal Write. Qual é a duração total da apresentação de dados válida para a memória?
 - Quantos estados de espera devemos inserir se a memória exigir a apresentação de dados válida por pelo menos 190 ns?
- 3.14** Um microprocessador tem uma instrução direta de incremento de memória, que soma 1 ao valor em um local da memória. A instrução tem cinco estágios: buscar do *opcode* (quatro ciclos de clock do barramento), buscar do endereço do operando (três ciclos), buscar do operando (três ciclos), soma de 1 ao operando (três ciclos) e armazenamento do operando (três ciclos).
- Por que quantidade (em percentagem) que a duração da instrução aumentará se tivermos que inserir dois estados de espera do barramento em cada operação de leitura e escrita de memória?
 - Repita considerando que a operação de incremento use 13 ciclos em vez de 3 ciclos.
- 3.15** O microprocessador Intel 8088 tem uma temporização do barramento de leitura semelhante à da Figura 3.19, mas requer quatro ciclos de clock do processador. Os dados válidos estão no barramento por uma quantidade de tempo que se estende para o quarto ciclo de clock do processador. Considere uma frequência do sinal de clock do processador de 8 MHz.
- Qual é a taxa máxima de transferência de dados?
 - Repita, mas considere a necessidade de inserir um estado de espera por byte transferido.
- 3.16** O Intel 8086 é um processador de 16 bits semelhante, de várias maneiras, ao 8088 de 8 bits. O 8086 utiliza um barramento de 16 bits que pode transferir 2 bytes de cada vez, desde que o byte de mais baixa ordem tenha um endereço par. Porém, o 8086 permite operandos de palavra com alinhamento par ou ímpar. Se uma palavra com alinhamento ímpar for referenciada, dois ciclos de memória, cada um consistindo em quatro ciclos de barramento, são necessários para transferir a palavra. Considere uma instrução no 8086 que envolva dois operandos de 16 bits. Quanto tempo é necessário para buscar os operandos? Dê a faixa de respostas possíveis. Considere uma frequência de sinal de clock de 4 MHz e nenhum estado de espera.
- 3.17** Considere um microprocessador de 32 bits cujo ciclo de barramento tenha a mesma duração de um microprocessador de 16 bits. Suponha que, na média, 20% dos operandos e instruções tenham 32 bits de tamanho, 40% tenham 16 bits de tamanho e 40% tenham 8 bits de tamanho. Calcule o ganho ao buscar instruções e operandos com o microprocessador de 32 bits.
- 3.18** O microprocessador do Problema 3.14 inicia o estágio de busca de operando da instrução de incremento direto da memória ao mesmo tempo em que um teclado ativa uma linha de requisição de interrupção. Depois de quanto tempo o processador entra no ciclo de processamento de interrupção? Considere uma frequência do sinal de clock do barramento de 10 MHz.
- 3.19** Desenhe e explique um diagrama de temporização para uma operação de escrita com um barramento PCI (semelhante à Figura 3.23).



Apêndice 3A Diagramas de temporização

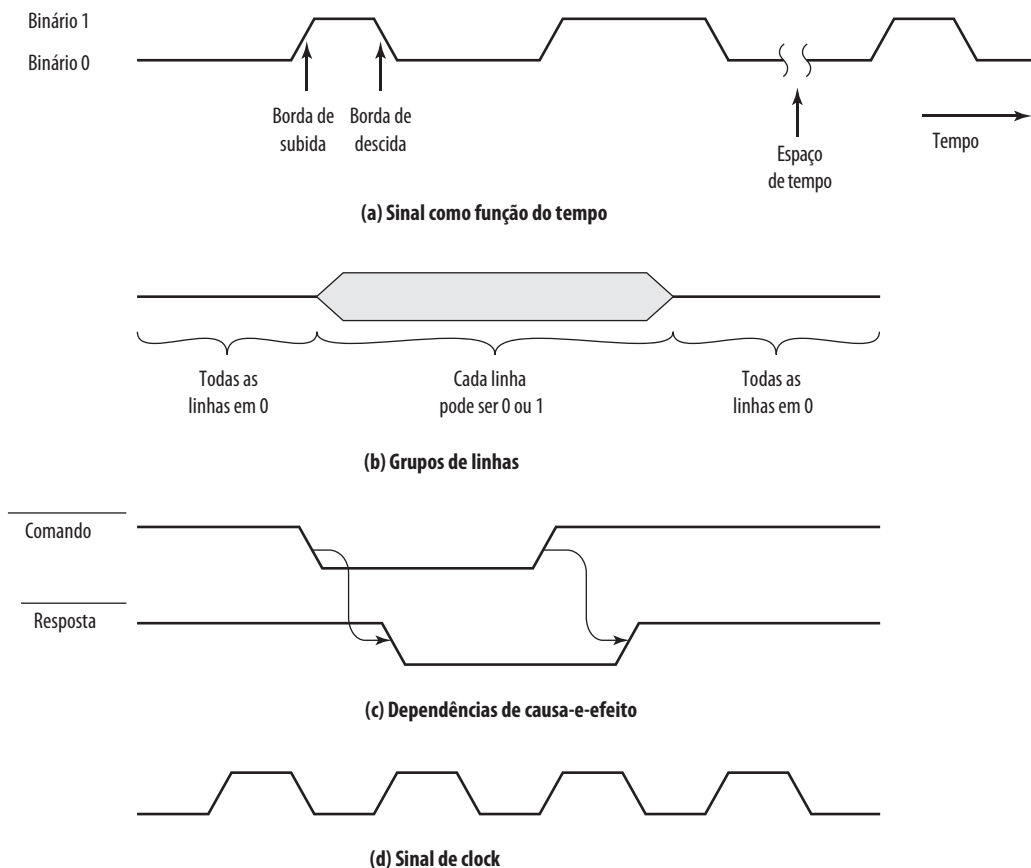
Neste capítulo, os diagramas de temporização são usados para ilustrar sequências de eventos e dependências entre eventos. Para o leitor não acostumado com diagramas de temporização, este apêndice oferece uma breve explicação.

A comunicação entre os dispositivos conectados a um barramento ocorre ao longo de um conjunto de linhas capazes de transportar sinais. Dois níveis de sinal (níveis de voltagem) diferentes, representando o binário 0 e o binário 1, podem ser transmitidos. Um diagrama de temporização mostra o nível de sinal em uma linha como uma função do tempo (Figura 3.27a). Por convenção, o nível de sinal binário 1 é representado como um nível mais alto que o do binário 0. Normalmente, o binário 0 é o valor padrão, ou seja, se nenhum dado ou outro sinal estiver sendo transmitido, então o nível em uma linha é aquele que representa o binário 0. Uma transição de sinal de 0 para 1 normalmente é referenciada como a *borda de subida* do sinal; uma transição de 1 para 0 é considerada uma *borda de descida*. Essas transições não são instantâneas, mas esse tempo de transição normalmente é pequeno em comparação com a duração de um nível de sinal. Por clareza, a transição normalmente é representada como uma linha inclinada, que exagera a quantidade de tempo relativa que a transição utiliza. Ocasionalmente, você verá diagramas que usam linhas verticais, o que incorretamente sugere que a transição é instantânea. Em um diagrama de temporização, pode acontecer que uma quantidade de tempo variável ou, pelo menos, irrelevante, se passe entre os eventos de interesse. Isso é representado por uma lacuna na linha de tempo.

Os sinais às vezes são representados em grupos (Figura 3.27b). Por exemplo, se os dados forem transferidos um byte de cada vez, então oito linhas são necessárias. Em geral, não é importante saber o valor exato sendo transferido em tal grupo, mas se os sinais estão presentes ou não.

Uma transição de sinal em uma linha pode disparar um dispositivo conectado para fazer mudanças de sinal em outras linhas. Por exemplo, se um módulo de memória detectar um sinal de controle de leitura (transição 0 ou 1), ele colocará sinais de dados nas linhas de dados. Esses

Figura 3.27 Diagramas de temporização



relacionamentos de causa e efeito produzem sequências de eventos. Setas são usadas nos diagramas de temporização para mostrar essas dependências (Figura 3.27c).

Na Figura 3.27c, a barra superior no nome do sinal indica que o sinal está ativo baixo, conforme mostrado. Por exemplo, $\overline{\text{Command}}$ está ativo em 0 volts. Isso significa que $\overline{\text{Command}} = 0$ é interpretado como o lógico 1, ou verdadeiro.

Uma linha de clock normalmente faz parte de um barramento do sistema. Um clock eletrônico está conectado à linha de clock e oferece uma sequência de transições repetitiva, regular (Figura 3.27d). Outros eventos podem ser sincronizados com o sinal de clock.

Referências

- a TANENBAUM, A. e WOODHULL, A. *Operating systems: design and implementation*. Upper Saddle River, NJ: Prentice Hall, 1997.
- b SHANLEY, T. e ANDERSON, D. *PCI systems architecture*. Richardson, TX: Mindshare Press, 1999.
- c ABBOT, D. *PCI bus demystified*. Nova York: Elsevier, 2004.